

UAM Social Network Login Config and API Integration Guide - 6.0.1-GA

Table of Contents

1. Preface - UAM Social Network Login Config and API Integration Guide.....	3
2. Introduction to UAM Social Network Login Config and API Integration.....	4
3. Social Network Login Configuration.....	9
3.1. Facebook Login.....	10
3.2. Google Login.....	16
3.3. LINE Login.....	21
3.4. WeChat Login.....	25
4. Social Network Login Using API.....	29
4.1. Social Network Identity Linking.....	30
4.2. Social Network Identity Unlinking.....	45
4.3. Social Network Login.....	49
4.4. Auto-Provisioning.....	62
4.5. Display Listing of User's Linked Social Network(s).....	66
5. Appendices.....	68
5.1. A. Allowing Email as User Id in AccessMatrix.....	69
5.2. B. Self-Service Linking Sample Program.....	70
5.3. C. Summary of Fields Required in Social Network Login.....	73

1. Preface - UAM Social Network Login Config and API Integration Guide

AccessMatrix Universal Access Management (UAM) supports the use of social network login with major social network platforms such as Facebook, Google, LINE, and WeChat. This document provides the configuration and integration steps necessary to use this feature.

Intended Reader

This guide is intended as a reference for security administrators and developers.

Document Scope

This document consists of a configuration guide and API integration details for using social network login with AccessMatrix UAM as the authentication backend server of the client application.

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- [UAM Administration Guide](#)
- [UAM Externalized Authorization, SAML, OAuth & OIDC API Reference](#)
- [AccessMatrix REST API Specification](#)

2. Introduction to UAM Social Network Login Config and API Integration

UAM supports social network login for Facebook, Google, LINE & WeChat. It enables the application to leverage on Social Network Login as user's authentication method with UAM as the authentication backend server. To support this feature, the user's social network login identity needs to be linked to the user's identity in AccessMatrix user's repository. Most of the time, the login flow of each social networks varies. It also depends on whether the social network users are the existing users of the client application.

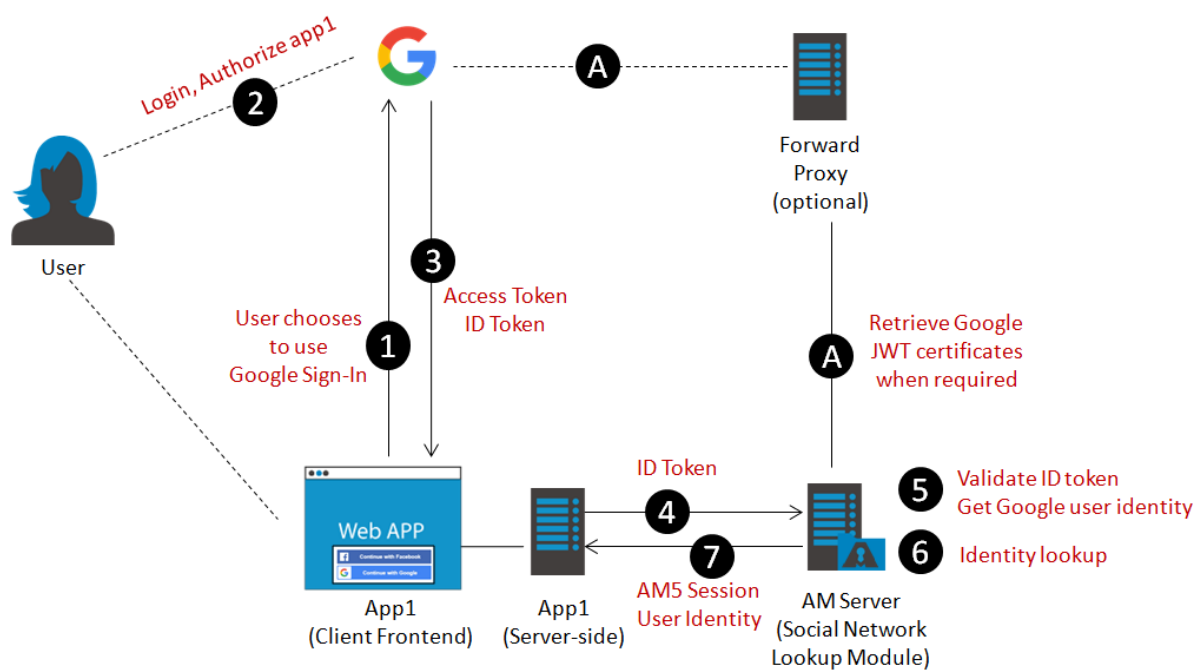
Social Network Login Flow

There are two common login flows: OAuth Implicit Flow and OAuth Authorization Code Flow. Facebook and Google provide both social network login flows while LINE & WeChat provide only one flow, i.e. using OAuth Authorization Code Flow.

OAuth Implicit Flow (Facebook/Google Client SDK):

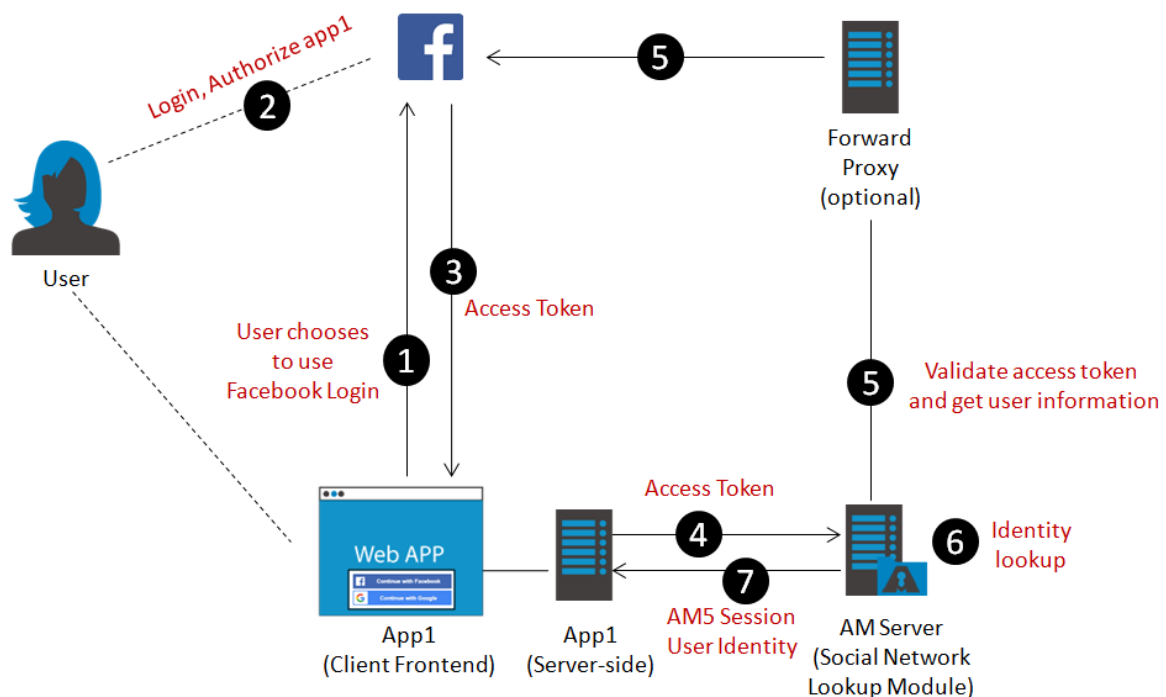
- This flow allows the application client-side to get the access token and the ID token directly from the social network.
- The access token and the ID token are to be sent to the application server-side so they can call the UAM server to verify and retrieve the user identity.
- Google Sign-In supports ID token. This allows the only ID token to be sent to the server-side for identity verification whereas the access token remains at the client-side.

Google Sign-In Client SDK (OAuth Implicit)



- Facebook does not support ID token. For Facebook, the access token needs to be given to the server-side for identity verification.

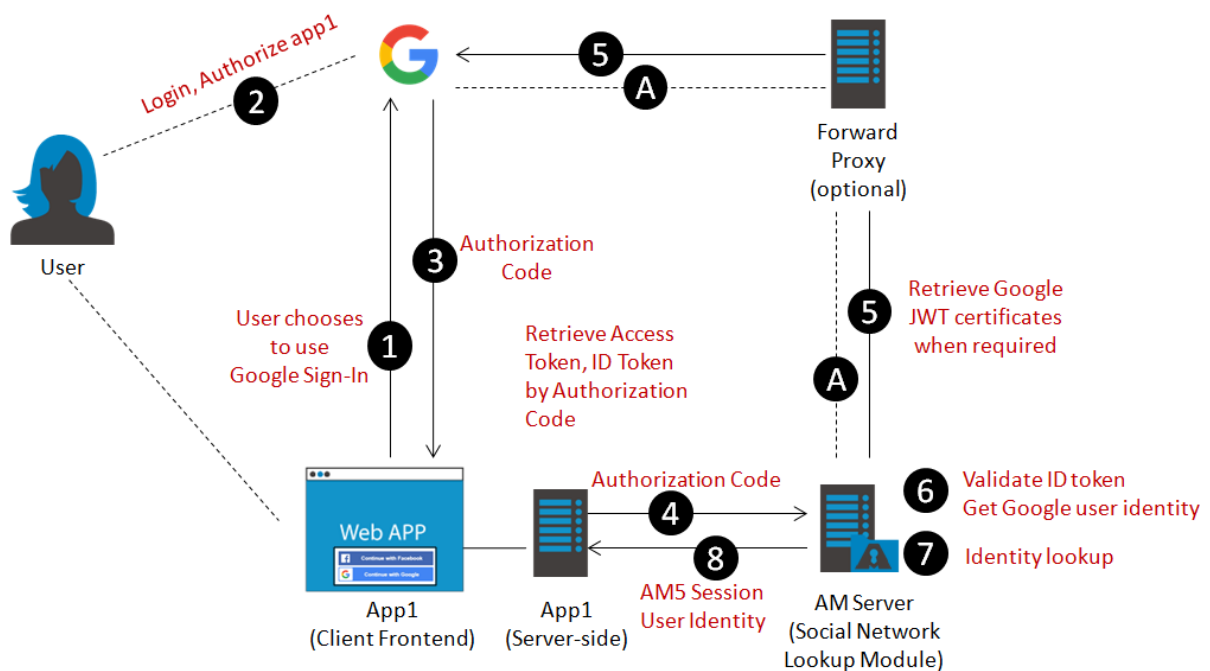
Facebook Client SDK (OAuth Implicit)



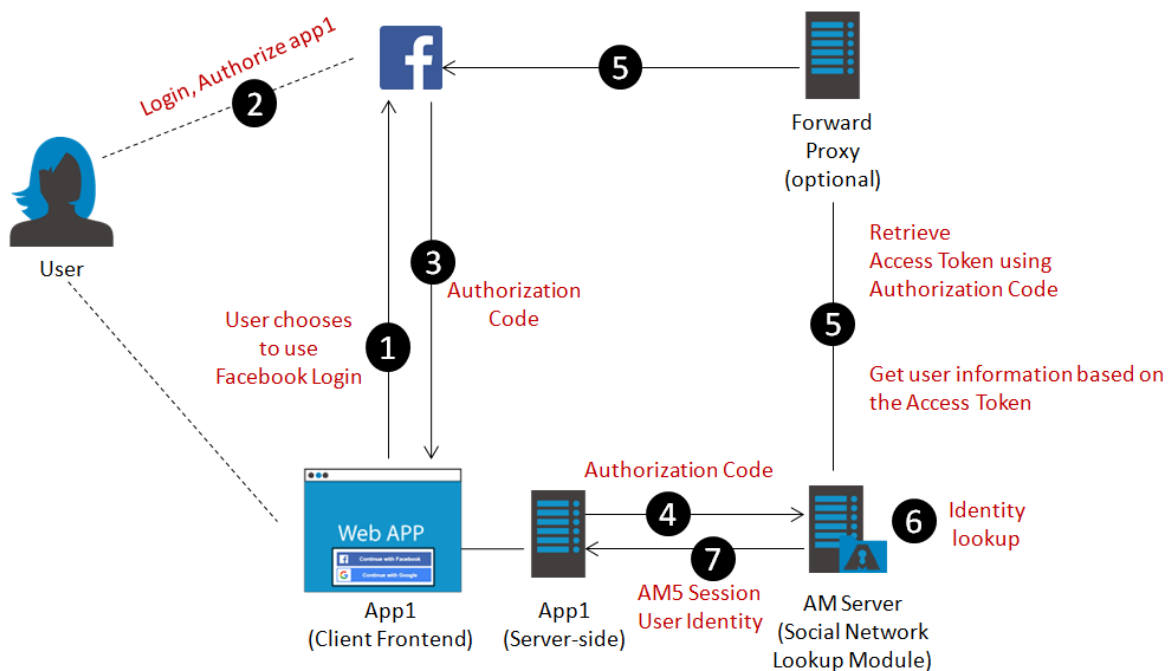
OAuth Authorization Code Flow:

- In Google Sign-In, it is called "OAuth 2.0 for Web Server Applications scenario". In Facebook, it is called "Manual Login Flow". In LINE, it is called "OAuth 2.0 authorization code grant flow and the OpenID Connect protocol". In WeChat, it is called "WeChat OAuth2.0 authorization login system".
 - This flow does not expose access token at the application client-side.
 - Facebook, Google, LINE or WeChat issues a one-time use authorization code at the client-side. This authorization code is sent to the application server-side so it can call the UAM server to verify and retrieve user identity.
 - UAM server calls Facebook, Google, LINE or WeChat to exchange the authorization code to get the access token and the ID token.
 - The purpose of this flow is not to expose the access token at the client-side.
-

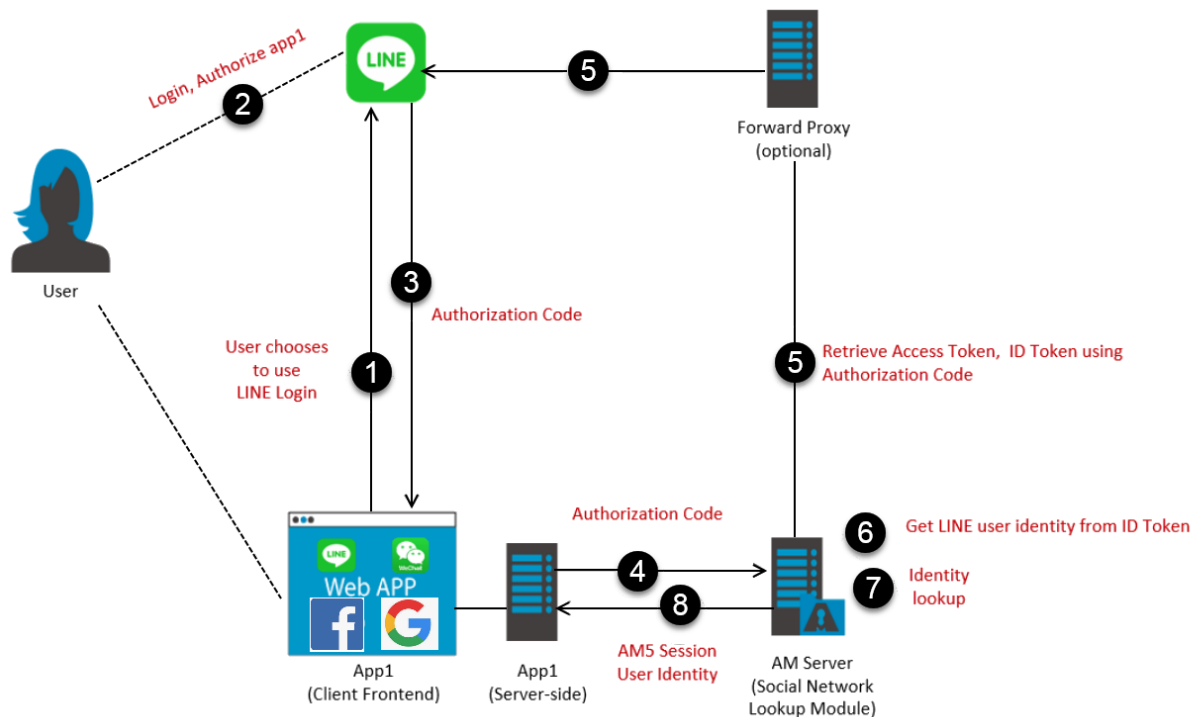
Google OAuth 2.0 for Web Server Applications (OAuth Authorization Code)



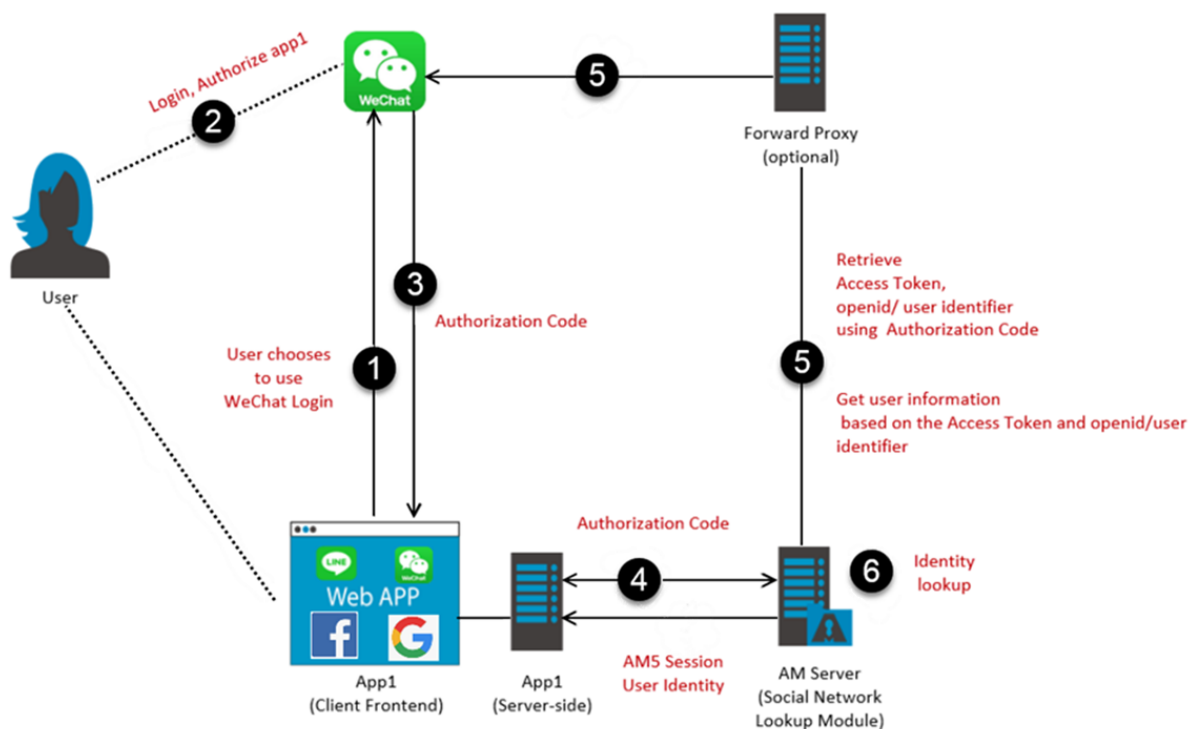
Facebook Manual Login (OAuth Authorization Code)



LINE OAuth 2.0 Authorization Code Grant Flow and the OpenID Connect Protocol (OAuth Authorization Code)



WeChat OAuth2.0 Authorization Login System (OAuth Authorization Code)



3. Social Network Login Configuration

The following sections describe the configurations required for setting up a social network login for their respective social network with AccessMatrix UAM server:

- [Facebook Login](#)
- [Google Login](#)
- [LINE Login](#)
- [WeChat Login](#)

3.1. Facebook Login

The section below explains how to configure the social network login for the Facebook application with the AccessMatrix UAM server.

Facebook Setup

The Facebook application has many features and settings. See below a brief summary of integration to the AccessMatrix UAM server. For more details, refer to [Facebook Developer documentation](#).

Two different flows of Facebook login:

- **Facebook Client SDK (using OAuth Implicit Flow)**

Access Token is acquired at the client-side and sent to the application's server-side for verification via the AccessMatrix UAM server.

i **Note:** Further references can be found under the **Web** section of the Facebook Developer documentation. (This reference is subject to change due to third party (Facebook) modification.)

- **Manual Login Flow (using OAuth Authorization Code Flow)**

Access Token is not exposed at the client-side. Authorization Code is used as an intermediary token to be exchanged for an access token. This exchange is done via the AccessMatrix UAM server.

i **Note:** Further reference can be found under the **Advanced - Manually Build a Login Flow** section of the Facebook Developer documentation. (This reference is subject to change due to third party (Facebook) modification.)

! **Important Note:** Go through the [Login Security](#) section of the Facebook Developer documentation for the security checklist of Facebook Login implementation for any project deployment.

Facebook Setup for Web Application

- **Prerequisite**

For the AccessMatrix system - Facebook Login integration, the following information is required:

- App ID (numerics, normally)
 - App Secret (only applicable for manual login flow)
 - App Redirect URI (only applicable for manual login flow)
-

- **Web Application Setup**

1. Create an **App** in the Facebook Developer Console.
2. Add the **Facebook Login** feature/product to the App.
3. Go to the **Facebook Login Settings** and carry out the following:
 - α. Enable **Client OAuth Login**, if not enabled.
 - β. Enable **Web OAuth Login**, if not enabled (only applicable for manual login flow).
 - χ. Add valid OAuth redirect URLs for the application and save the changes.
4. Go to the **App Settings**.
5. Find the **App ID** and **Secret**.

Admin Console Configuration

Perform the following steps to integrate Facebook login with AccessMatrix UAM system:

1. Navigate to **Administration Dashboard > Configuration > Authentication > User Lookup Module**, click **Create**, and then select "Facebook Login" under **Implementation** to create a Facebook lookup module (refer to [Facebook Login Lookup Module](#) section for the attributes descriptions). Click **Next**.
 2. Specify the **User Lookup Module Id** and **User Lookup Module Name**.
 3. Provide the **Facebook's App Id** and **App Secret** based on the values generated or as shown on the Facebook Developer Console.
 4. If the manual login flow is used, set **Use Authorization Code Flow** to "Yes".
 5. Set **Return Access Token in Response** to "Yes" to return the access token to the application (API caller) and to allow the application to use the access token to call Facebook API. The access token life cycle such as "refresh" is handled by the application.
 6. AccessMatrix server needs to call Facebook API via the internet. Therefore, the HTTP forward proxy must be configured.
 - a. Enable the HTTP Proxy by setting **Enabled** to "true".
 - b. Configure the Proxy Server: **Proxy Host Name** and **Port**.
 - c. Configure the HTTP Proxy Server Authentication Scheme:
 - For Basic Authentication, configure **User Name** and **User Password**.
-

-
- For NTLM Authentication, set **NTLM Authentication** to "true". It is mandatory to specify **Domain Name**. In addition to that, configure the AM5 server's host domain name or IP address in **amsystem.properties** with the property key "am.social.ntlm.host".
7. Choose whether to allow auto-provisioning, i.e. allow a user to be automatically created in AccessMatrix Repository by setting the **Enabled** attribute under **Auto Create New User** to "Yes". These Facebook user profile attributes will be mapped to the user as follows:
 - Choose either "Facebook ID" or "Facebook Email" from **User ID Mapping** to be mapped to the AccessMatrix UAM user Id.
 - "Facebook Email" is the default ID to be mapped. When "Facebook Email" is specified as the **User ID Mapping** value, the user's email must be registered on the Facebook user profile, otherwise, AccessMatrix system will trigger an error response for the missing email.
 - When "Facebook ID" is specified as the **User ID Mapping** value, then AccessMatrix system will map Facebook Email to AccessMatrix User ID if it is available. Facebook ID will be used to map to AccessMatrix User ID only when user's Email is not registered.
 - The requester APP can also override the **User ID Mapping** setting by providing "userId" as an additional param in the API call.
 - The Facebook user name will be mapped to the AccessMatrix system user name.
 8. Save the changes.
 9. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**, click **Create**, and then select "Social Network Login" under **Implementation** to create a Social Network Login module.
 10. Specify the **Login Module Id** and **Login Module Name**.
 11. Select its **Login Policy** and its **Authentication Level** in the **Configuration** tab.
 12. Save the changes.
 13. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm** and click **Create** to create an authentication realm.
 14. Specify the **Authentication Realm Id** and **Authentication Realm Name**.
 15. Specify the lookup module and login module created in previous steps.
-

16. Save the changes.

Facebook Login Lookup Module

This section contains the descriptions of Facebook's login lookup module attributes.

Attribute Name	Description
*User Lookup Module Id	Unique Id of lookup module.
*User Lookup Module Name	Unique name of lookup module.
Description	Description of lookup module.
Version	Lookup module entry version.
Implementation	Type of lookup module implementation, i.e. "Facebook Login".
Attributes Tab	
App Id	Unique Id of application. This value is generated from Facebook Developer Console.
App Secret	Secret code of application. This value is generated from Facebook Developer Console.
Use Authorization Code Flow	Indicate whether an input is an Authorization Code or Access Token. - Setting to "Yes" will use authorization code flow. Authorization code flow is used for application with server-side component and when access token is not exposed to the client-side. - Setting to "No" will not use authorization code flow, therefore input must be an access token. Default: false
User fields to Query	Returns the queried user field, e.g. "Id", "name", or "email". Default: id, name, email
Return Access Token in Response	Determine whether to return access token to the AccessMatrix client-side that calls API to login via Facebook. Default: No
HTTP Proxy	

Attribute Name	Description
Enabled	Setting to "true" will enable the HTTP Proxy. Default: false
Proxy Host Name	Refers to the URL of the proxy server.
Port	Refers to the port of the proxy server.
User Name	Refers to the user name used to authenticate the proxy server.
User Password	Refers to the user's password used to authenticate the proxy server.
NTLM Authentication	Indicates whether NTLM authentication is used. If NTLM Authentication is enabled, Domain Name must be specified. In addition to that, it might be necessary to configure the AM5 server host domain name or IP address in amsystem.properties with the property key "am.social.ntlm.host" for each instance. Default: false
Domain Name	Refers to the domain name of AD used in NTLM authentication.
HTTP Reverse Proxy	
Replace Facebook URL's host name	Several Facebook URLs will be triggered during runtime. By default, the hostname of those URLs is "https://graph.facebook.com". If a different hostname is specified (e.g. "http://myforwardProxyhost"), then it will replace the hostname of the corresponding Facebook URLs. Default: https://graph.facebook.com
Auto Create New User	
Enabled	Setting this to enabled will map: "email" (user profile attribute from Facebook) to user Id and user's email "name" (user profile attribute from Facebook) to user name Default: false
User ID Mapping	- If "Facebook Email" is specified, user's Facebook account is expected to be registered with email as his or her primary Facebook Contact or else an exception will be thrown to request users to do so. - If "Facebook ID" is specified as User Id Mapping, user's Facebook Email, which is retrieved from user's Facebook Profile, will be mapped as User ID during auto user creation process, however, if Facebook Email is not available, Facebook ID will be used instead. Default: Facebook Email
User Store	Refers to the target JDBC user store. Default: DefaultStore

3.2. Google Login

This section provides details on the social network login configuration for Google application.

Google Setup

Google app has many features and settings for different platforms. See below a brief summary of its integration with the AccessMatrix server. For more information, refer to [Google Developer documentation](#).

Two different flows of Google login:

Google Sign-In SDK (using OAuth Implicit flow)

- ID Tokens are acquired at the client-side and sent to the application's server-side for verification via AM5 server.
- It is optional to send access token from the client-side and to the application's server-side for verification via AccessMatrix UAM server. If the access token is provided to the AccessMatrix UAM server, AccessMatrix UAM server will validate the association between ID token and access token.



Note: More references can be found under the **Customize the Sign-In Button** section of Google Developer documentation. (This reference is subject to change due to third party (Google) modification.)

OAuth 2.0 for Web Server Applications Scenario (using OAuth Authorization Code flow)

- Access Token is not exposed at the client-side. The Authorization Code is used as an intermediary token to be exchanged for an access token. This exchange is done via AccessMatrix UAM server.



Note: More references can be found under the **Google Sign-In for server-side apps** section of Google Developer documentation. (This reference is subject to change due to third party (Google) modification.)

Setup for Web Application

- **Prerequisite**

For the AccessMatrix - Google Login integration to work, the following information is required:

- Client ID
-

-
- URI origin(s) of the JavaScript client application
 - Client Secret (only applicable for "OAuth 2.0 for Web Server Applications" scenario)
 - Redirect URI (only applicable for "OAuth 2.0 for Web Server Applications" scenario)

- **Web Application Setup**

Below are the steps to integrate Google Sign-In to AccessMatrix server. For additional information, refer to [Google Sign-In documentation for Websites](#).

1. Create an **App** in the Google Cloud Platform Console.
2. Choose **OAuth client** and **Web Application** as part of the App settings.
 - α. Configure origin URI of the JavaScript client app.
 - β. Configure the redirect URI of the client app (only applicable for "OAuth 2.0 for Web Server Applications" scenario).
 - χ. Once done, the **App ID** and **Secret** will be provided.

Admin Console Configuration

Perform the following steps to integrate Google sign-in with AM5 server:

1. Navigate to **Administration Dashboard > Configuration > Authentication > User Lookup Module**, click **Create** and select "Google Sign-In Lookup" under **Implementation** (refer to [Google Sign-In Lookup Module](#) section for the attributes descriptions).
 2. Specify the **User Lookup Module Id** and **User Lookup Module Name**.
 3. Provide Google's **Client Id** and **Client Secret** based on the values generated or as shown on Google Cloud Platform Console.
 4. If the "OAuth 2.0 for Web Server Applications" scenario is used, set **Use Authorization Code Flow** to "Yes".
 5. Choose whether to allow auto-provisioning, i.e. allows the user to be automatically created in AccessMatrix Repository by setting **Auto Create New User** to "Yes". These Google user profile attributes will be mapped to the user as follows:
 - Google user email will be mapped to the AM5 user Id and user email.
 - Google user name will be mapped to the AM user name.
-

-
6. Set **Return Access Token in Response** to "Yes" to return the access token to the application (API caller) and to allow the application to use the access token to call Google API. The access token life cycle such as "refresh" is handled by the application.
 7. AM5 server needs to call Google API via the internet. Therefore, the HTTP forward proxy must be configured.
 - a. Enable the HTTP Proxy by setting **Enabled** to "true".
 - b. Configure the Proxy Server: **Proxy Host Name** and **Port**.
 - c. Configure the HTTP Proxy Server Authentication Scheme:
 - For Basic Authentication, configure **User Name** and **User Password**.
 - For NTLM Authentication, enable **NTLM Authentication**. It is mandatory to specify **Domain Name**. In addition to that, it may be needed to configure the AM5 server host domain name or IP address in **amsystem.properties** with the property key "am.social.ntlm.host".
 8. Save the changes.
 9. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**, click **Create** and select "Social Network Login" under **Implementation**.
 10. Specify the **Login Module Id** and **Login Module Name**.
 11. Select its **Login Policy** and its **Authentication Level** in the **Configuration** tab.
 12. Save the changes.
 13. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm** and create an authentication realm.
 14. Specify the **Authentication Realm Id** and **Authentication Realm Name**.
 15. Then, specify the lookup module and login module created in previous steps.
 16. Save the changes.

Google Sign-In Lookup Module

This section contains the descriptions of Google's Sign-In lookup module attributes.

Attribute Name	Description
*User Lookup Module Id	Unique Id of lookup module.

Attribute Name	Description
*User Lookup Module Name	Unique name of lookup module.
Description	Description of lookup module.
Auto Create New User	Setting to "Yes" will allow social network users to be created in AM5 automatically if the users do not exist in AM5 yet. The new user will be created in Default Store of AM5. Default: No
Version	Lookup module entry version.
Implementation	Type of lookup module implementation, i.e. "Google Sign-In".
User Store	Refers to the target JDBC user store. Default: DefaultStore
Attributes Tab	
Client Id	Unique Id of a client. This value is generated from Google Cloud Platform Console.
Client Secret	Secret code of client. This value is generated from Google Cloud Platform Console.
Use Authorization Code Flow	Indicate whether an input is an Authorization Code or ID Token. - Setting to "Yes" will use the authorization code flow. Authorization code flow is used for application with the server-side component and when the access token is not exposed to the client-side. - Setting to "No" will not use authorization code flow, therefore input must be an ID token. Default: false
Return Access Token in Response	Determine whether to return access token. Default: No
HTTP Proxy	
Enabled	Setting to "true" will enable the HTTP Proxy. Default: false
Proxy Host Name	Refers to the URL of the proxy server.
Port	Refers to the port of the proxy server.

Attribute Name	Description
User Name	Refers to the user name used to authenticate the proxy server.
User Password	Refers to the user's password used to authenticate the proxy server.
NTLM Authentication	Indicates whether NTLM authentication is used. If NTLM Authentication is enabled, Domain Name must be specified. In addition to that, it might be necessary to configure the AM5 server host domain name or IP address in amsystem.properties with the property key "am.social.ntlm.host" for each instance. Default: false
Domain Name	Refers to the domain name of AD used in NTLM authentication.
HTTP Reverse Proxy	
Replace Google Account's URL hostname	Several Google Account URLs will be triggered during runtime. By default, the hostname of the URLs is "https://accounts.google.com". If a different hostname is specified (e.g. "http://myforwardProxyhost"), then it will replace the hostname of the corresponding Google URLs. Default: https://accounts.google.com
Replace Google API's URL hostname	Several Google API URLs will be triggered during runtime. By default, the hostname of the URLs is "https://www.googleapis.com". If a different hostname is specified (e.g. "http://myforwardProxyhost"), then it will replace the hostname of the corresponding Google URLs. Default: https://www.googleapis.com

3.3. LINE Login

This section provides details on the social network login configuration for LINE application.

LINE Setup

LINE application has many features and settings. See below a brief summary of its integration to the AccessMatrix server. For more information, refer to [LINE Developer documentation](#).

The flow of LINE login:

OAuth 2.0 authorization code grant flow and the OpenID Connect protocol

- Access Token is not exposed at the client-side. An Authorization Code is used as an intermediary token to be exchanged for an access token. This exchange is done via the AccessMatrix server.



Note: More references can be found under the **Web - Integrating LINE Login** section of the LINE Developer documentation. (This reference is subject to change due to third party (LINE) modification.)

LINE Setup for Web Application

- **Prerequisite**

For the AccessMatrix - LINE Login integration to work, the following information is required:

- App ID
- App Secret
- App Redirect URI

- **Web Application Setup**

1. Create an **Application Provider** in LINE Developer Console.
2. Create an **Application Channel** under the created Application Provider.
3. Go to **LINE Login Settings** and do the following:
 - Add valid OAuth redirect URIs for the application and save changes.
4. Go to **Channel Settings**.
5. **App ID** and **Secret** can be found here.

Admin Console Configuration

Perform the following steps to integrate Google sign-in with AM5 server:

1. Navigate to **Administration Dashboard > Configuration > Authentication > User Lookup Module**, click **Create** and select "LINE Sign-In Lookup" under **Implementation** (refer to [LINE Sign-In Lookup Module](#) section for the attributes descriptions).
 2. Specify the **User Lookup Module Id** and **User Lookup Module Name**.
 3. Provide the LINE's **Client Id** (App Id) and **Client Secret** (App Secret) based on the values generated or as shown on the LINE Developer Console.
 4. Choose whether to allow auto-provisioning, i.e. allow a user to be automatically created in AccessMatrix Repository by setting the **Enabled** attribute under **Auto Create New User** to "Yes". These LINE user profile attributes will be mapped to the user as follows:
 - LINE user email will be mapped to AM5 user Id
 - LINE user name will be mapped to AM user name
 5. Set **Return Access Token in Response** to "Yes" to return the access token to the application (API caller), to allow the application to use the access token to call LINE API. The access token life cycle, such as refresh, is handled by the application.
 6. AccessMatrix server needs to call LINE API via the internet. Therefore, the HTTP forward proxy must be configured.
 - a. Enable the HTTP Proxy by setting **Enabled** to "true".
 - b. Configure the Proxy Server: **Proxy Host Name** and **Port**.
 - c. Configure the HTTP Proxy Server Authentication Scheme:
 - For Basic Authentication, configure **User Name** and **User Password**.
 - For NTLM Authentication, enable **NTLM Authentication**. It is mandatory to specify **Domain Name**. In addition to that, it may be needed to configure the AM5 server host domain name or IP address in **amsystem.properties** with the property key "am.social.ntlm.host".
 7. Save the changes.
 8. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**, click **Create** and select "Social Network Login" under **Implementation**.
-

-
9. Specify the **Login Module Id** and **Login Module Name**.
 10. Select its **Login Policy** and its **Authentication Level** in the **Configuration** tab.
 11. Save the changes.
 12. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm** and create an authentication realm.
 13. Specify the **Authentication Realm Id** and **Authentication Realm Name**.
 14. Then, specify the lookup module and login module created in previous steps.
 15. Save the changes.

LINE Sign-In Lookup Module

This section contains the descriptions of LINE's Sign-In lookup module attributes.

Attribute Name	Description
*User Lookup Module Id	Unique Id of lookup module.
*User Lookup Module Name	Unique name of lookup module.
Description	Description of lookup module.
Version	Lookup module entry version.
Implementation	Type of lookup module implementation, i.e. "LINE Sign-In".
Attributes Tab	
Client Id	Unique Id of client. This value is generated from LINE Developer Console.
Client Secret	Secret code of client. This value is generated from LINE Developer Console.
User Fields to Query	Returns the queried user field. "sub" represents the User ID generated by LINE. Default: sub,name
Return Access Token in Response	Determine whether to return access token. Default: No
HTTP Proxy	
Enabled	Setting to "true" will enable the HTTP Proxy. Default: false

Attribute Name	Description
Proxy Host Name	Refers to the URL of the proxy server.
Port	Refers to the port of the proxy server.
User Name	Refers to the user name used to authenticate the proxy server.
User Password	Refers to the user's password used to authenticate the proxy server.
NTLM Authentication	Indicates whether NTLM authentication is used. If NTLM Authentication is enabled, Domain Name must be specified. In addition to that, it might be necessary to configure the AM5 server host domain name or IP address in amsystem.properties with property key "am.social.ntlm.host" for each instance. Default: false
Domain Name	Refers to the domain name of AD used in NTLM authentication.
Auto Create New User	
Enabled	Setting this to enabled will map: "email" (user profile attribute from LINE) to user Id and user's email "name" (user profile attribute from LINE) to user name Default: false
User Store	Refers to the target JDBC user store. Default: DefaultStore

3.4. WeChat Login

This section provides details on the social network login configuration for WeChat application.

WeChat Setup

WeChat Application has many features and settings. See below a brief summary of its integration to the AccessMatrix server. For more information, refer to [WeChat Developer documentation](#).

Flow of WeChat login:

WeChat OAuth 2.0 authorization login system

- Access Token is not exposed at the client-side. An Authorization Code is used as an intermediary token to be exchanged for an access token. This exchange is done via the AccessMatrix server.



Note: More references can be found under the **WeChat Login** section of the WeChat Developer documentation. (This reference is subject to change due to third party (WeChat) modification.)

WeChat Setup for Web Application

- **Prerequisite**

For AccessMatrix - WeChat Login integration to work, the following information is required:

- App ID
- App Secret
- App Redirect URI

- **Web Application Setup**

1. Create an **App** in the WeChat Admin Center in the Developer Console.
2. Go to **WeChat Login Settings** and do the following:
 - Add Authorization callback domain and save changes.
3. Go to **App Details**.
4. **App ID** and **Secret** can be found here.

Admin Console Configuration

Perform the following steps to integrate WeChat sign-in with AM5 server:

-
1. Navigate to **Administration Dashboard > Configuration > Authentication > User Lookup Module**, click **Create**, and select "WeChat Sign-In Lookup" under **Implementation** (refer to [WeChat Sign-In Lookup Module](#) section for the attributes descriptions).
 2. Specify the **User Lookup Module Id** and **User Lookup Module Name**.
 3. Provide the WeChat's **Client Id** (App Id) and **Client Secret** (App Secret) based on the values generated or as shown on Google Cloud Platform Console.
 4. If "OAuth 2.0 for Web Server Applications" scenario is used, set **Use Authorization Code Flow** to "Yes".
 5. Choose whether to allow auto-provisioning, i.e. allow a user to be automatically created in AccessMatrix Repository by setting **Auto Create New User** to "Yes". These WeChat user profile attributes will be mapped to the user as follows:
 - WeChat user Id mapped to AM5 user Id.
 - WeChat nickname mapped to AccessMatrix username.
 6. Set **Return Access Token in Response** to "Yes" to return the access token to the application (API caller) and to allow the application to use the access token to call WeChat API. The access token life cycle such as "refresh" is handled by the application.
 7. AccessMatrix server needs to call WeChat API via the internet. Therefore, the HTTP forward proxy must be configured.
 - a. Enable the HTTP Proxy by setting **Enabled** to "true".
 - b. Configure the Proxy Server: **Proxy Host Name** and **Port**.
 - c. Configure the HTTP Proxy Server Authentication Scheme:
 - For Basic Authentication, configure **User Name** and **User Password**.
 - For NTLM Authentication, enable **NTLM Authentication**. It is mandatory to specify **Domain Name**. In addition to that, it may be needed to configure the AM5 server host domain name or IP address in **amsystem.properties** with property key "am.social.ntlm.host".
 8. Save the changes.
 9. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**, click **Create** and select "Social Network Login" under **Implementation**.
 10. Specify the **Login Module Id** and **Login Module Name**.
 11. Select its **Login Policy** and its **Authentication Level** in the **Configuration** tab.
-

-
12. Save the changes.
 13. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm** and create an authentication realm.
 14. Specify the **Authentication Realm Id** and **Authentication Realm Name**.
 15. Then, specify the lookup module and login module created in previous steps.
 16. Save the changes.

WeChat Sign-In Lookup Module

This section contains the descriptions of WeChat's Sign-In lookup module attributes.

Attribute Name	Description
*User Lookup Module Id	Unique Id of lookup module.
*User Lookup Module Name	Unique name of lookup module.
Description	Description of lookup module.
Auto Create New User	Setting to "Yes" will allow social network users to be created in AM5 automatically if the users do not exist in AM5 yet. The new user will be created in Default Store of AM5. Default: No
Version	Lookup module entry version.
Implementation	Type of lookup module implementation, i.e. WeChat Sign-In.
User Store	Refers to the target JDBC user store. Default: DefaultStore
Attributes Tab	
Client Id	Unique Id of client. This value is generated from WeChat Developer Console.
Client Secret	Secret code of client. This value is generated from WeChat Developer Console.
Return Access Token in Response	Determine whether to return access token. Default: No
HTTP Proxy	

Attribute Name	Description
Enabled	Setting to "true" will enable the HTTP Proxy. Default: false
Proxy Host Name	Refers to the URL of the proxy server.
Port	Refers to the port of the proxy server.
User Name	Refers to the user name used to authenticate the proxy server.
User Password	Refers to the user's password used to authenticate the proxy server.
NTLM Authentication	Indicates whether NTLM authentication is used. If NTLM Authentication is enabled, Domain Name must be specified. In addition to that, it might be necessary to configure the AM5 server host domain name or IP address in amsystem.properties with property key "am.social.ntlm.host" for each instance. Default: false
Domain Name	Refers to the domain name of AD used in NTLM authentication.

4. Social Network Login Using API

Use cases of client application login flow:

- **Social Network Identity Linking**
 - For client application which users exist in AccessMatrix user repository can make use of [Self-Service Identity Linking](#) feature where:
 - users can authenticate itself with the existing 1FA, then do the social network sign-in prior to performing the self-service linking with AccessMatrix.
 - For client application which users have yet to be provisioned can make use of [Auto-Provisioning](#) feature where:
 - upon first login via Social Network Login, user's external identity will be provisioned to AccessMatrix user repository and implicitly linked to the newly-created user.
- [Social Login](#)
- [Unlink Social Identity Mapping in AM5](#)
- [Display listing of User's Linked Social Network](#)

4.1. Social Network Identity Linking

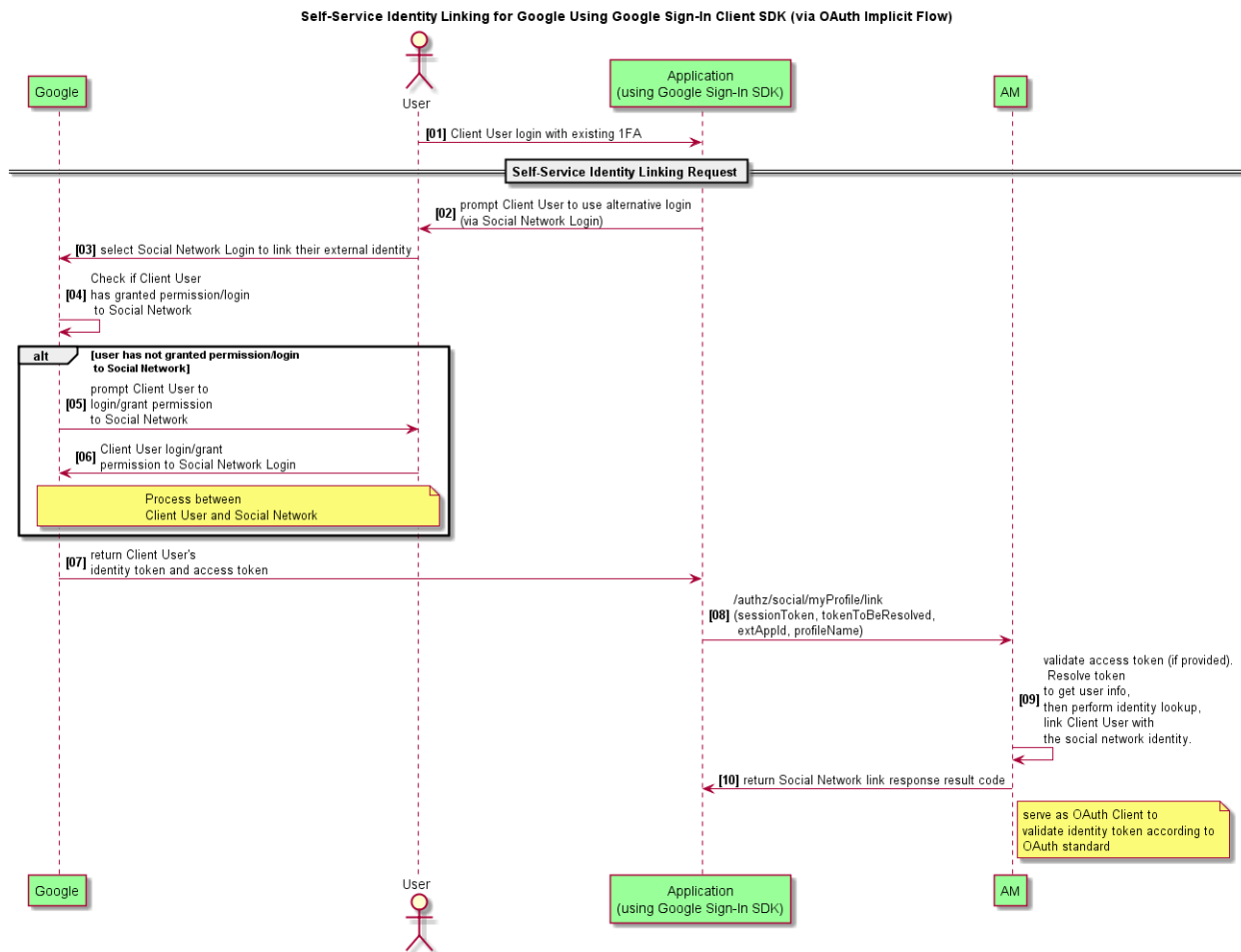
Self-service identity linking process requires the user to be authenticated (i.e., it has a valid AM5 session) using another method. The social network can be done by calling the `/authz/social/myProfile/link` API to get the user's social network login identity linked up with AccessMatrix user's identity. Different social network applications might have different data flows to achieve single sign-on. Below are the sequence diagrams and sample codes that illustrate Identity Linking flow of the supported social network applications.

Google

The process of identity mapping will link up the user's social network login identity with AccessMatrix user's identity, and the corresponding Google Sign-In Lookup Module. Email is the user's Google sign-in identity and it is globally unique across Google's third-party applications.

Using Google Sign-In Client SDK (via OAuth Implicit Flow)

Sequence Diagram



REST Sample Code

This is a sample REST API call for Google self-service identity linking using Google Sign-In Client SDK (via OAuth Implicit Flow).

Link Social Network User Profile Client details for Google	
URL: /authz/social/myProfile/link	
Request	
JSON	
{ "extAppId": "google", "tokenToBeResolved":	

Link Social Network User Profile Client details for Google

```
"eyJhbGciOiJSUzI1NiIsImtp....<truncated in
sample>.....9bRH9vE5-
jVWHFoBGS163VXfgzxU0U_Zm9g12fk__FSzBHxX3_jRSi5f0NXJw",
  "profileName": "profile1",
  "sessionToken":
"10001_ELns3HSnKM3Kju9RuGk8pqp....<truncated in
sample>.....a-
7HCiSnx0xhlGuIjd8YO8bg3HLWHP6yxvDLH6L0OKXKkDsqtg"
}
```

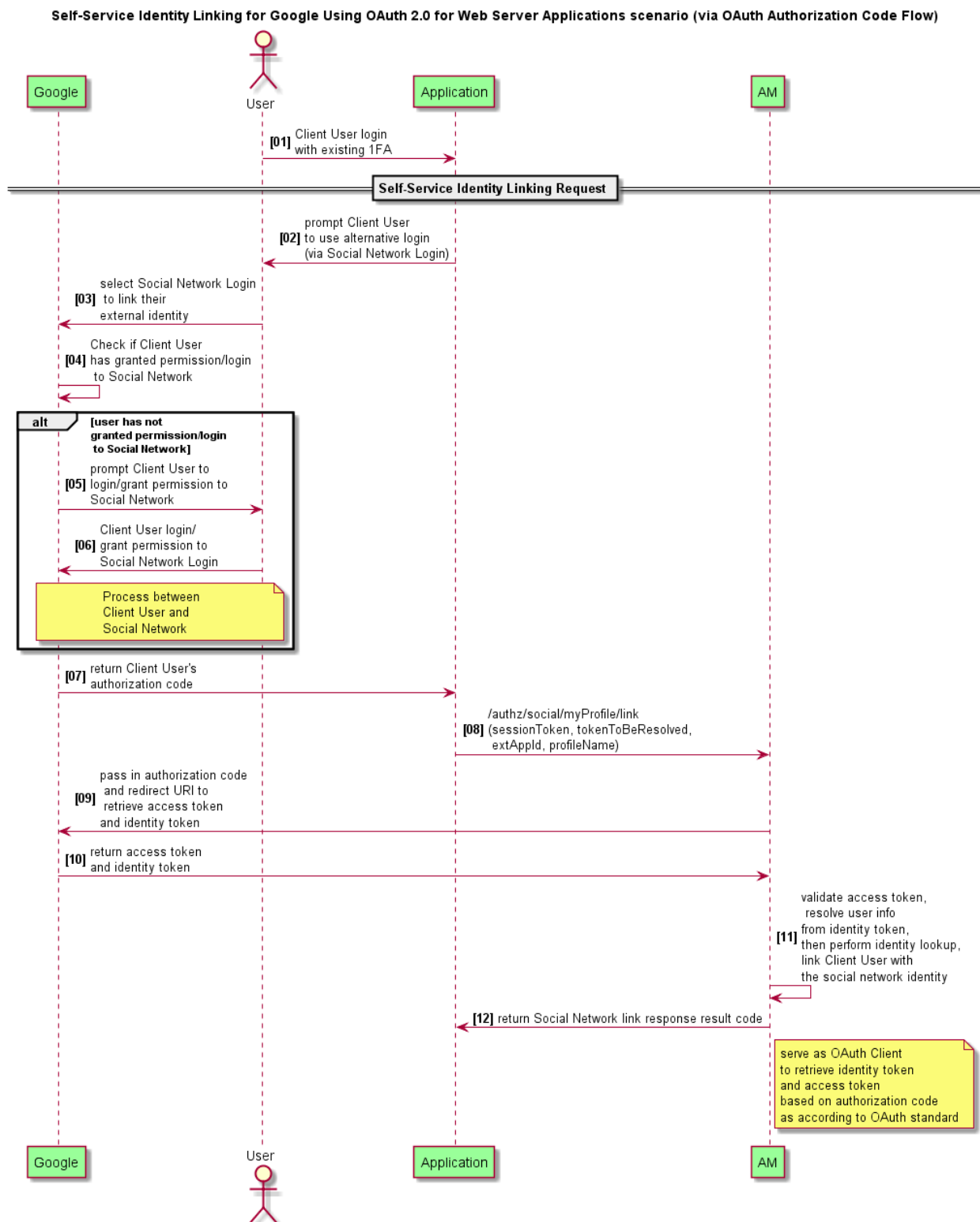
Response

JSON

```
{
  "result": {
    "externalId": "External Social Network Application
User ID",
    "attributes": {...}
  }
}
```


OAuth 2.0 for Web Server Applications Scenario (via OAuth Authorization Code Flow)

Sequence Diagram



REST Sample Code

This is a sample REST API call for Google self-service identity linking using OAuth 2.0 for Web Server Applications Scenario (via OAuth Authorization Code Flow).

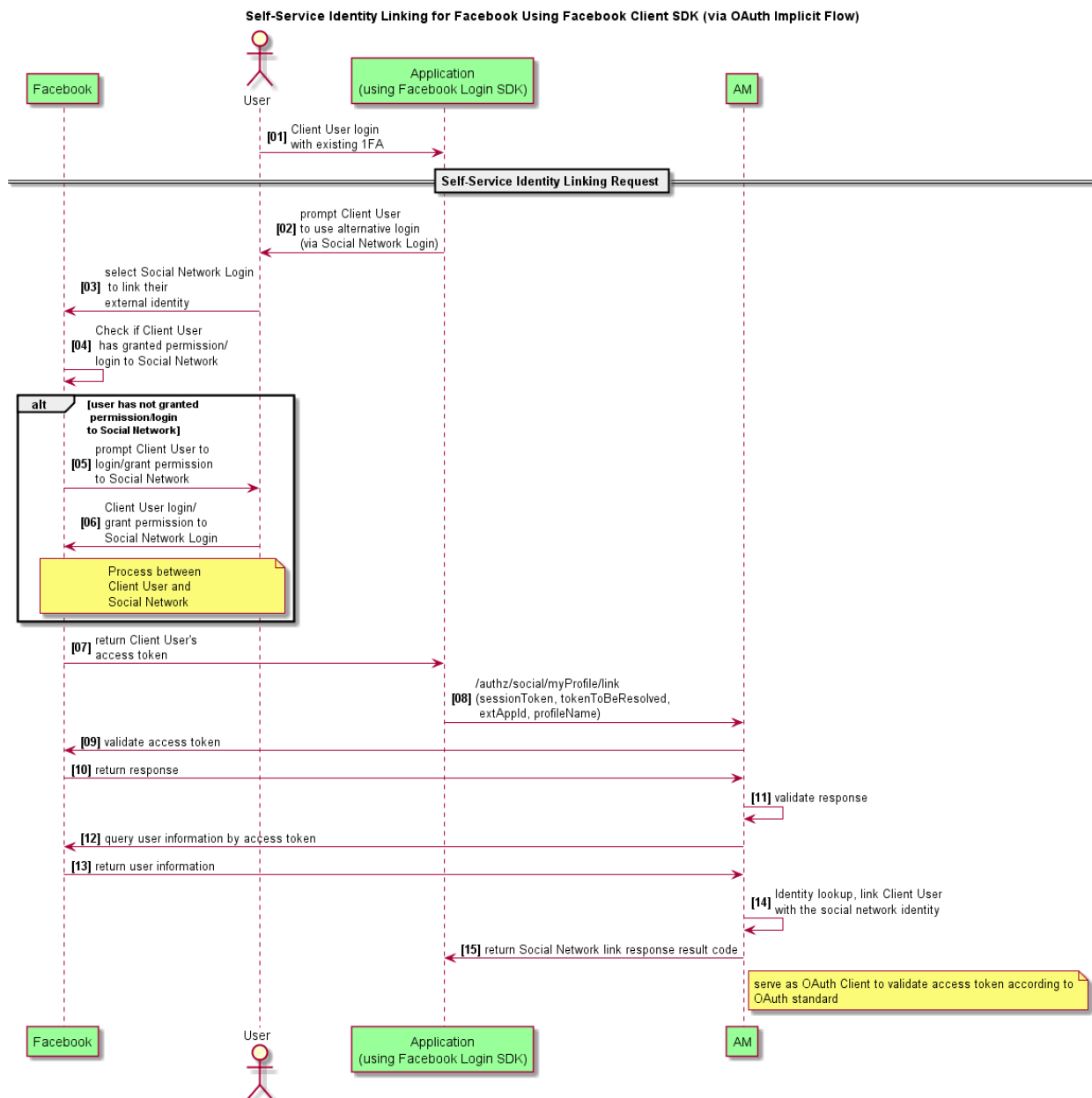
Link Social Network User Profile Client details for Google	
URL: /authz/social/myProfile/link	
Request	
JSON	<pre>{ "extAppId": "google", "tokenToBeResolved": "4/ud4OC5fRI5cmWvkej.....<truncated in sample>.....gwTK4zL8igI9kvISvTylSA", "profileName": "profile1", "sessionToken": "10001_ELns3HSnKM3Kju9RuGk8pqp.....<truncated in sample>.....a-7HCiSnx0xhlGuIjd8YO8bg3HLWHP6yxvDLH6L0OKXKkDsqtg", "params": {"redirect_uri": "http://dev.i-sprint.demo.com:8081/"} }</pre>
Response	
JSON	<pre>{ "result": { "externalId": "External Social Network Application User ID", "attributes": {...} } }</pre>

Facebook

The process of identity mapping will link up the user's social network login identity with AccessMatrix user's identity, and the corresponding Facebook Log-In Lookup Module. Facebook Id is user's Facebook log-in identity and it is app-scoped and unique for the particular Facebook's third-party application.

Using Facebook Client SDK (via OAuth Implicit Flow)

Sequence Diagram



REST Sample Code

This is a sample REST API call for Facebook self-service identity linking using Facebook Client SDK (via OAuth Implicit Flow).

Link Social Network User Profile Client details for Facebook

URL: /authz/social/myProfile/link

Request

JSON

```
{
  "extAppId": "facebook",
  "tokenToBeResolved": "EAAFB0VqRiCABALik0i.....<truncated
in sample>.....kjBZCeFp6",
  "profileName": "profile1",
  "sessionToken":
"10001_Elns3HSnKM3Kju9RuGk8pqp.....<truncated in
sample>.....a-
7HCiSnx0xhlGuIjd8YO8bg3HLWHP6yxvDLH6L0OKXKkDsqtg"
}
```

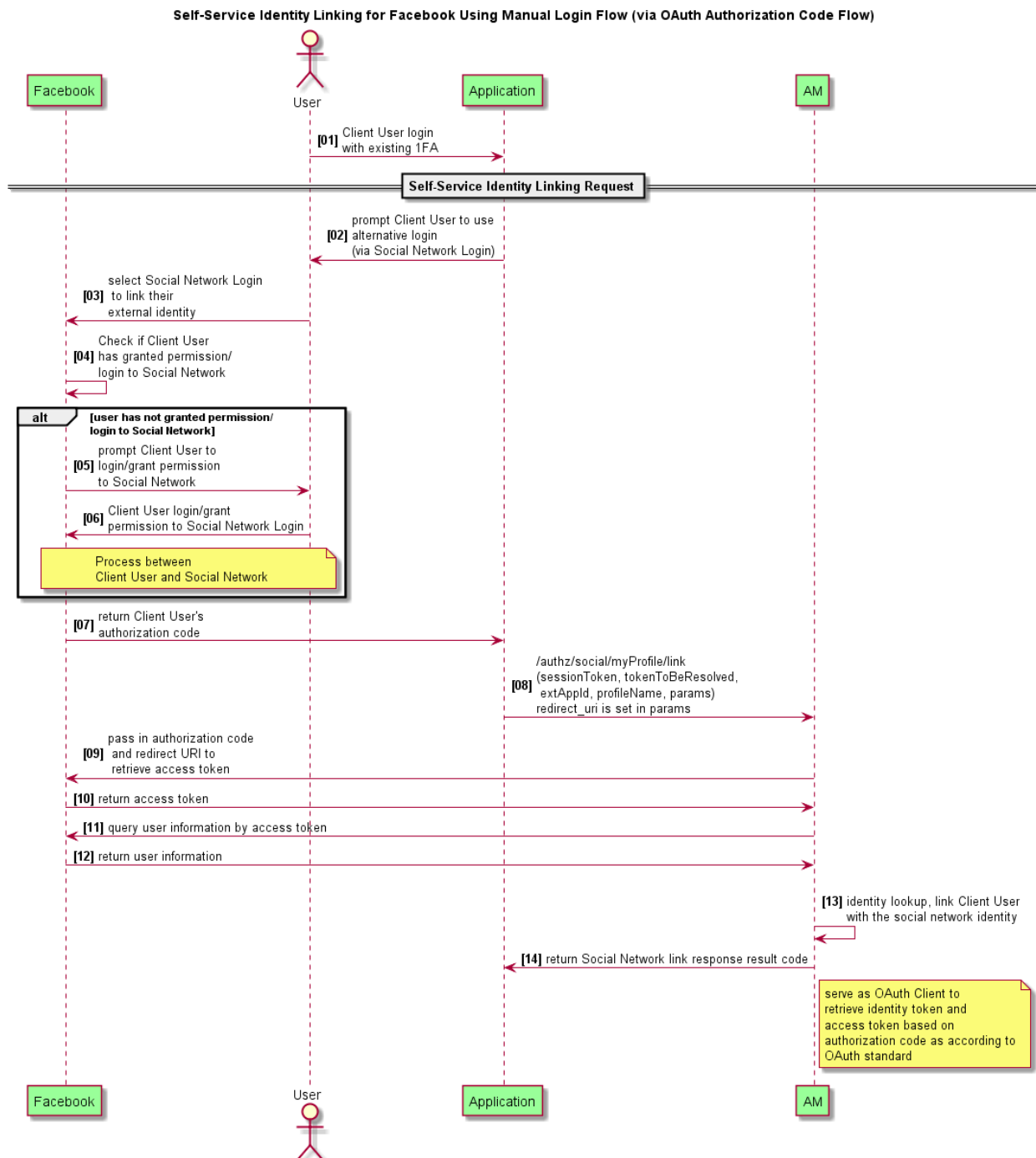
Response

JSON

```
{
  "result": {
    "externalId": "External Social Network Application
User ID",
    "attributes": {...}
  }
}
```

Manual Login Flow (via OAuth Authorization Code Flow)

Sequence Diagram



REST Sample Code

This is a sample REST API call for Facebook self-service identity linking using Manual Login Flow (via OAuth Authorization Code Flow).

Link Social Network User Profile Client details for Facebook	
URL: /authz/social/myProfile/link	
Request	
JSON	
<pre>{ "extAppId": "google", "tokenToBeResolved": "AQA4Tamq_WfGyaf-31i8nRZxyd.....<truncated in sample>.....0eR3sGE6bbk6GvWnpcdKCECf#_=_", "profileName": "profile1", "sessionToken": "10001_ELns3HSnKM3Kju9RuGk8pqp.....<truncated in sample>.....a-7HCiSnx0xhlGuIjd8YO8bg3HLWHp6yxvDLH6L0OKXKkDsqtg", "params": {"redirect_uri": "http://fb.i-sprint.demo:8081/fbdemo/"} }</pre>	
Response	
JSON	
<pre>{ "result": { "externalId": "External Social Network Application User ID", "attributes": {...} } }</pre>	

LINE

The process of identity mapping would link up the user's social network login identity with AccessMatrix user's identity, and the corresponding LINE Log-In Lookup Module. LINE sub ("sub is User Id wrapped within idToken JWT") is user's LINE log-in identity and it is unique across the LINE's channels ("Channel is the application") under a LINE provider ("Provider is the entity(individual or company) that offer apps").

Below is the sample screenshot for reference (where i-Sprint is a LINE provider with few registered channels):

Provider

i-sprint.innovation



Sort by date



LINE Login



Messaging API

Global Auth 2 i-Sprint

LINE Login

globalAuth2



LINE Login



globalAuth



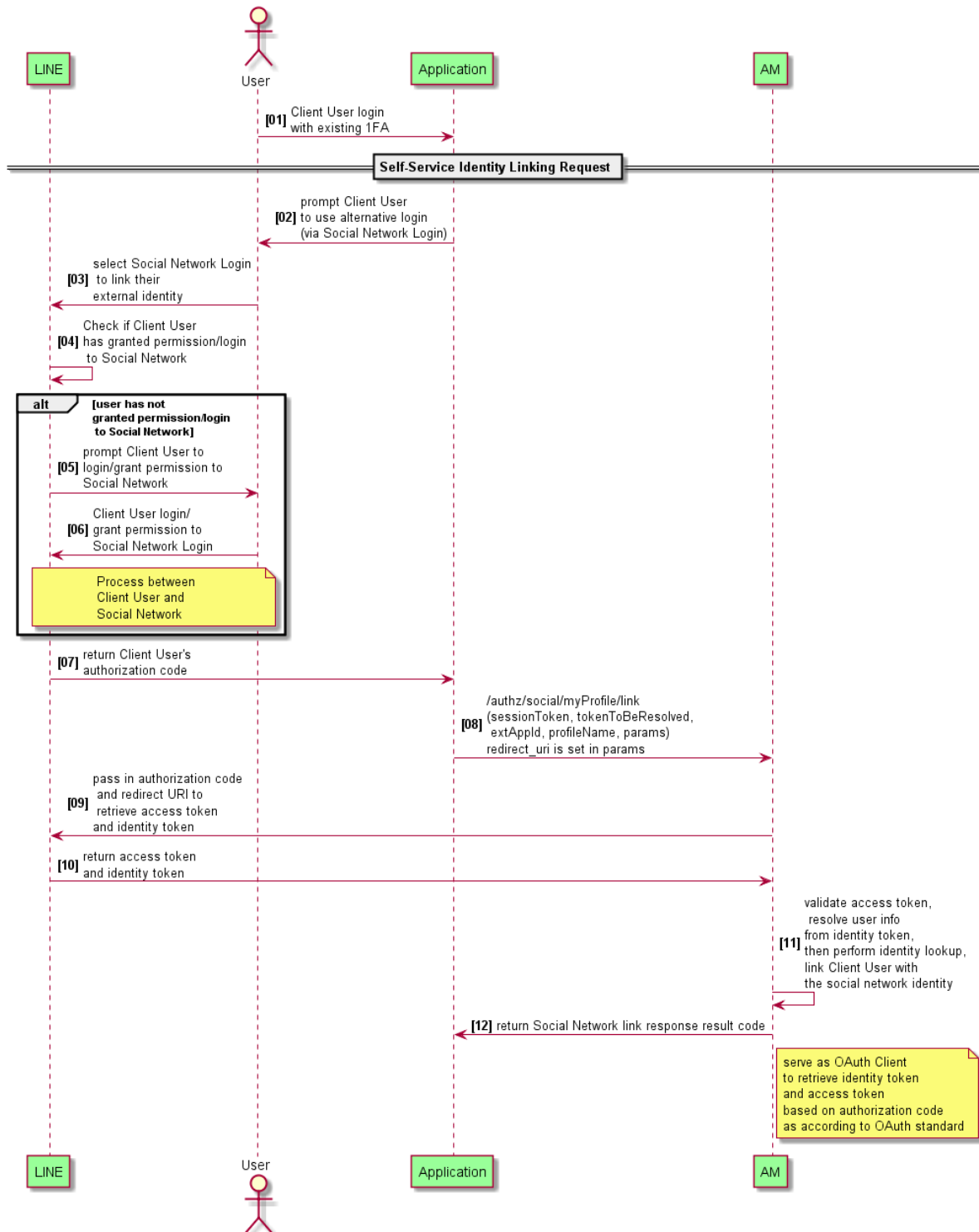
LINE Login



OAuth 2.0 Authorization Code Grant Flow and the OpenID Connect Protocol (via OAuth Authorization Code Flow)

Sequence Diagram

Self-Service Identity Linking for LINE OAuth 2.0 authorization code grant flow and the OpenID Connect protocol (via OAuth Authorization Code Flow)



REST Sample Code

This is a sample REST API call for LINE self-service identity linking using OAuth 2.0 authorization code grant flow and the OpenID Connect protocol (via OAuth Authorization Code Flow).

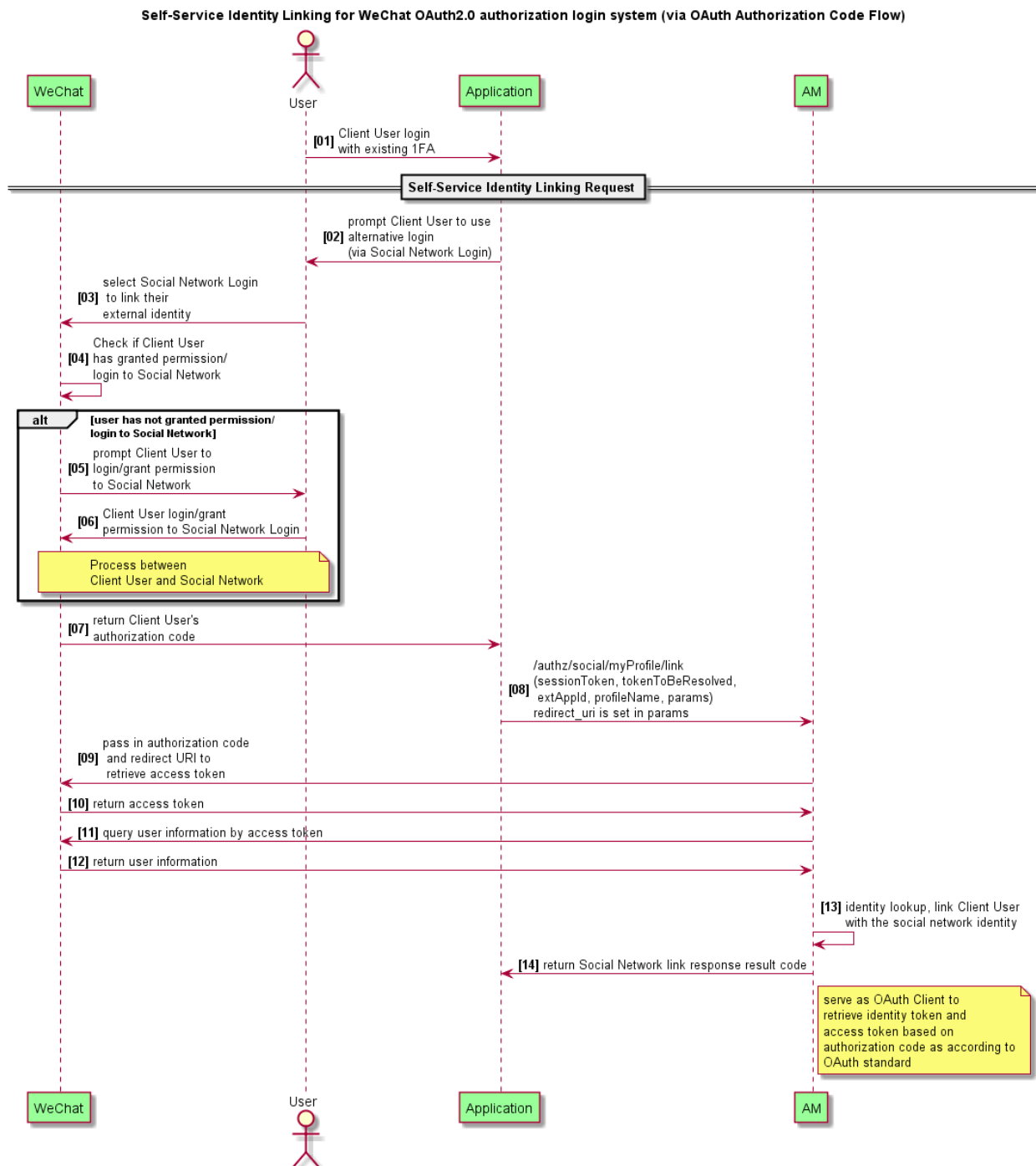
Link Social Network User Profile Client details for Google	
URL: /authz/social/myProfile/link	
Request	
JSON	<pre>{ "extAppId": "line", "tokenToBeResolved": "4/ud4OC5fRI5cmWvkej.....<truncated in sample>.....gwTK4zL8igI9kvISvTylSA", "profileName": "profile1", "sessionToken": "10001_ELns3HSnKM3Kju9RuGk8pqp.....<truncated in sample>.....a-7HCiSnx0xhlGuIjd8YO8bg3HLWHP6yxvDLH6L0OKXKkDsqtg", "params": {"redirect_uri": "http://dev.i-sprint.demo.com:8081/linedemo"} }</pre>
Response	
JSON	<pre>{ "result": { "externalId": "External Social Network Application User ID", "attributes": {...} } }</pre>

WeChat

The process of identity mapping will link up the user's social network login identity with AccessMatrix user's identity, and the corresponding WeChat Log-In Lookup Module. WeChat UnionId is user's WeChat log-in identity and it is unique across the WeChat's third-party applications under one WeChat Open Platform Account.

OAuth 2.0 Authorization Login System (via OAuth Authorization Code Flow)

Sequence Diagram



REST Sample Code

This is a sample REST API call for WeChat self-service identity linking using OAuth 2.0 authorization login system (via OAuth Authorization Code Flow).

Link Social Network User Profile Client details for Google	
URL: /authz/social/myProfile/link	
Request	
JSON	
<pre>{ "extAppId": "WeChat", "tokenToBeResolved": "AQA4Tamq_WfGyaf-31i8nRZxyd.....<truncated in sample>.....0eR3sGE6bbk6GvWnpCdKCECf#_=_", "profileName": "profile1", "sessionToken": "10001_Elns3HSnKM3Kju9RuGk8pqp.....<truncated in sample>.....a-7HCiSnx0xhlGuIjd8YO8bg3HLWHp6yxvDLH6L0OKXKkDsQtg", "params": {"redirect_uri": "http://fb.i-sprint.demo:8081/wcdemo/"} }</pre>	
Response	
JSON	
<pre>{ "result": { "externalId": "External Social Network Application User ID", "attributes": {...} } }</pre>	

Parameters & Exceptions

This method is used to link AM5 user identity with Social Network user identity:

- tokenToBeResolved is a token given by Social Network upon successful login (mandatory field).
- socialNetworkLookupId is an ID of Social Network Lookup module, e.g. Facebook Lookup Module, Google Lookup Module, LINE Lookup Module and WeChat Lookup Module (mandatory field).
- profileName is the profile name used to identify the identity mapping of user and external Social Network ID (mandatory field).
- params is the map for extra fields to be passed in.

For exception and other details of the API method, refer to UAM Externalized Authorization & SAML API Reference document.

[Click here](#) to view a summary of fields specifically required in different implementations of Facebook ,Google, LINE & WeChat.

Responses

Returns a JSON object from this API. It is consist of:

- **external Id** - the identifier of Social Network login user used by AccessMatrix for identity linking.
- **attributes:**
 - the token(s), returned by social network app upon successful login to resolve user info, and some other attribute(s), returned by social network app after resolving and validating the token.
 - Below are the attributes/fields of each social network app:
 - **Google:**
 - token - access_token, id_token,
 - attributes - email, sub, name, hd, picture, locale
 - **Facebook:**
 - token - access_token
 - attributes - email, id, name
 - **LINE:**
 - token - access_token, id_token,
 - attributes - iss, sub, aud, exp, iat, nonce, name, picture, email
 - **WeChat:**
 - token - access_token,
 - attributes - openid, nickname, sex, province, city, country, headimgurl, privilege, unionid

4.2. Social Network Identity Unlinking

Identity Unlinking is a process of unlinking Social Network Identity from AccessMatrix user. This operation may be performed via the agent/admin or self-service method.

- **Self-service method**

It requires a user to be authenticated, where the user's session token will be used to call the `/authz/social/myProfile/unlink` API to get its social network identity unlinked from the AccessMatrix user.

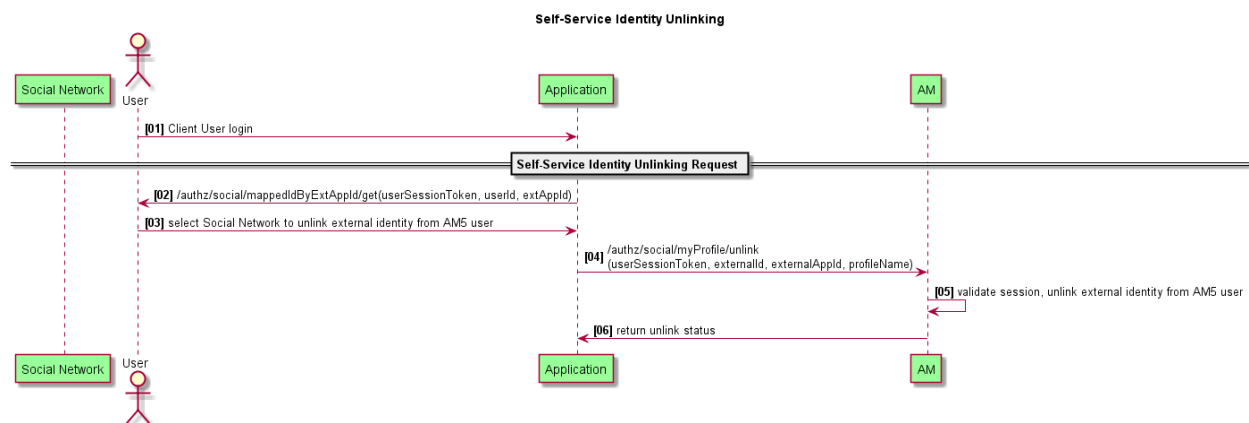
- **Agent/admin method**

This method allows administrators or help desk personnel to unlink AccessMatrix users from their social network identity using the `/authz/social/user/unlink` API.

Below are the sequence diagrams and sample codes that illustrate Identity Unlinking of the supported social network applications:

Self-Service

Sequence Diagram



[Click here](#) to find out more about the `/authz/social/mappedIdByExtAppId/get` API.

REST Sample Code

This is a sample REST API call of self-service identity unlinking.

Unlink Social Network User Profile Client details	
URL: /authz/social/myProfile/unlink	
Request	
JSON	<pre>{ "extAppId": "facebook", "externalId": "user_email@facebook.com", "profileName": "profile1", "sessionToken": "10001_Elns3HSnKM3Kju9RuGk8pqp.....<truncated in sample>.....a- 7HCiSnx0xhlGuIjd8YO8bg3HLWHP6yxvDLH6L0OKXKkDsqtg" }</pre>
Response	
JSON	<pre>{ "result": true }</pre>

Parameters & Exceptions

This method is to be called with end-user login session to unlink mapping of user's Social Network identity from AM5 user:

- externalId is a unique identifier of Social Network login user used for unlinking the mapping.
- extAppId is the ID of Social Network Login Lookup module, e.g. Facebook Lookup Module, Google Lookup Module, LINE Lookup Module and WeChat Lookup Module (mandatory field).
- profileName is the profile name used to identify the identity mapping of user and external Social Network ID.

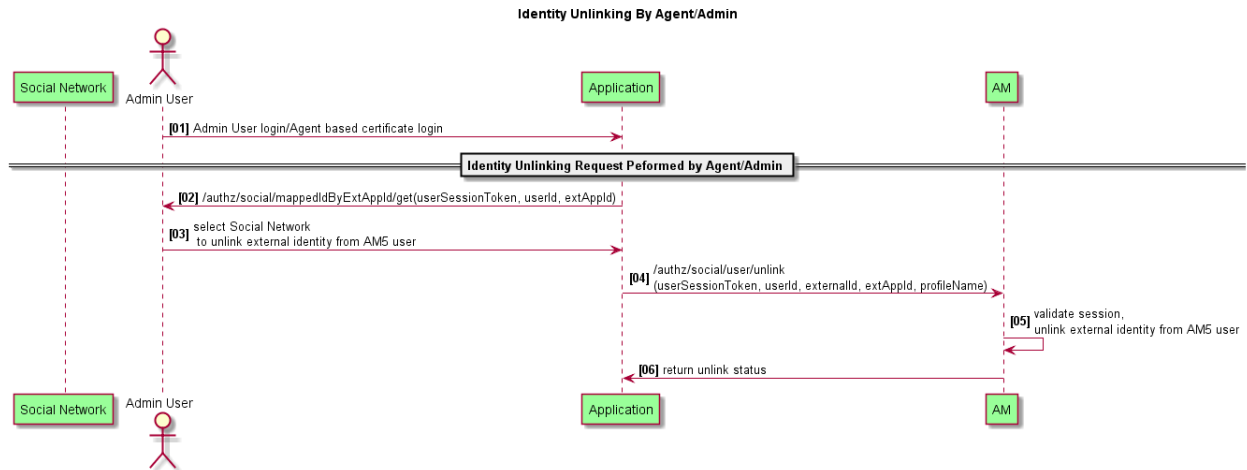
For exception and other details of the API method, refer to the *UAM Externalized Authorization & SAML API Reference*.

Response

The method returns Unlinking status in boolean, e.g. "true", if successful.

By Agent/Admin

Sequence Diagram



[Click here](#) to find out more about the /authz/social/mappedIdByExtAppId/get API.

REST Sample Code

This is a sample REST API call of identity unlinking by agent/admin.

Link Social Network User Account Client details by admin	
URL: /authz/social/user/unlink	
Request	
JSON	
<pre>{ "extAppId": "google", "externalId": "108158203193729", "profileName": "profile1", "userId": "demouser", "sessionToken": "10001_ELns3HSnKM3Kju9RuGk8pqp.....<truncated in sample>.....a- 7HCiSnx0xhlGuIjd8YO8bg3HLWHp6yxvDLH6L0OKXKkDsqtg" }</pre>	
Response	
JSON	

Link Social Network User Account Client details by admin

```
{  
  "result": true  
}
```

Parameters & Exceptions

This method is used to unlink mapping of Social Network user identity from AccessMatrix user:

- `userId` is the user found in AccessMatrix repository. (mandatory field).
- `externalId` is a unique identifier of Social Network login user used for unlinking the mapping.
- `extAppId` is the ID of Social Network Login Lookup module, e.g. Facebook Lookup Module, Google Lookup Module, LINE Lookup Module & WeChat Lookup Module (mandatory field).
- `profileName` is the profile name used to identify the identity mapping of user and external Social Network ID.

For this method, either the `userId` or `userUUID_eq` field needs to be provided.

For exception and other details of the API method, refer to the *UAM Externalized Authorization & SAML API Reference*.

Response

- The method returns Unlinking status in boolean, e.g. "true" if successful.
-

4.3. Social Network Login

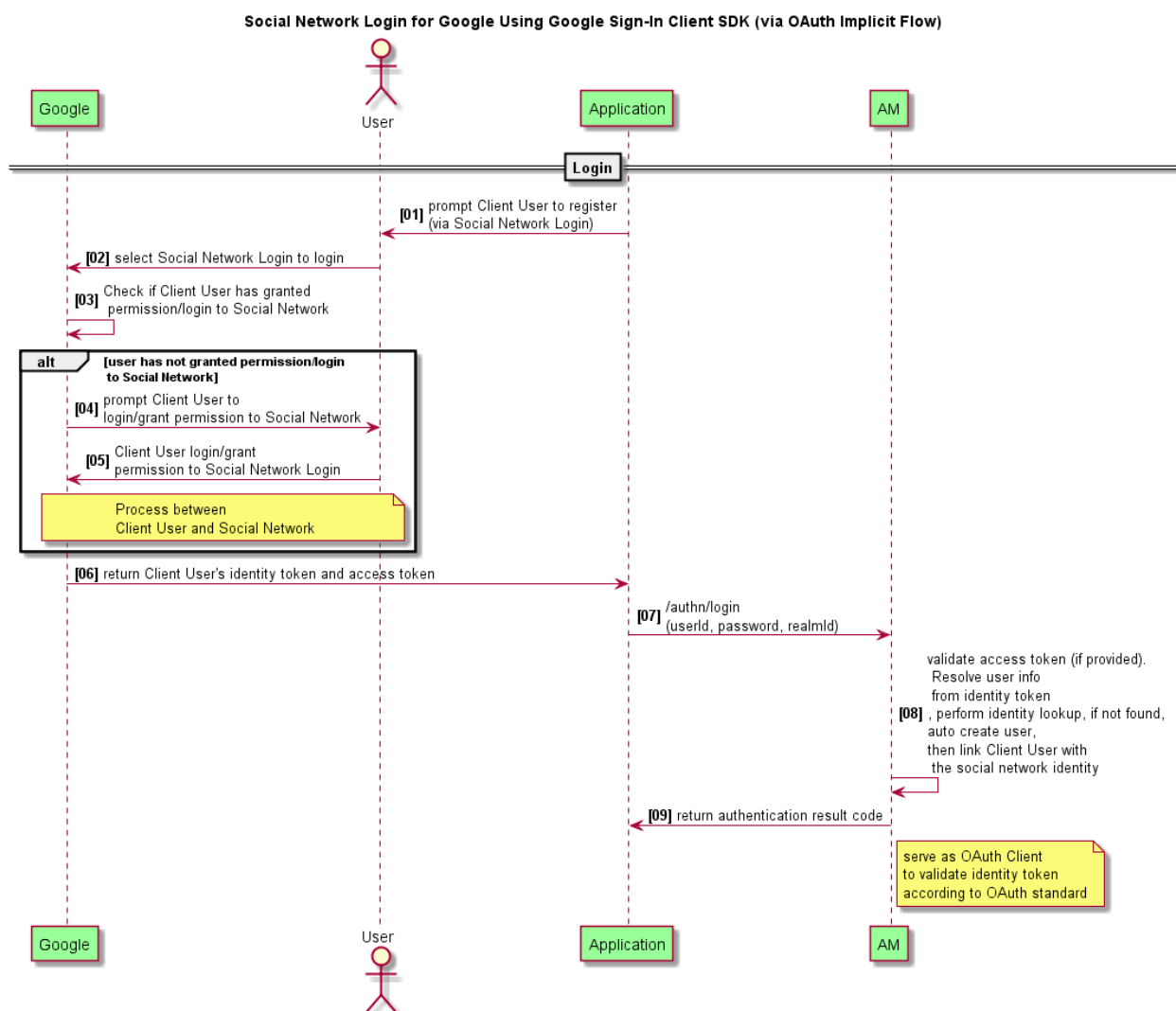
Social Network Login is a form of single sign-on using existing information from a social networking service such as Facebook, Google, LINE, or WeChat to sign into a third party website instead of creating a new login account specifically for that website.

Below are the data flow diagrams and sample codes that illustrate Social Network Login of the supported social network applications.

Google

Using Google Sign-In SDK (via OAuth Implicit Flow)

Sequence Diagram



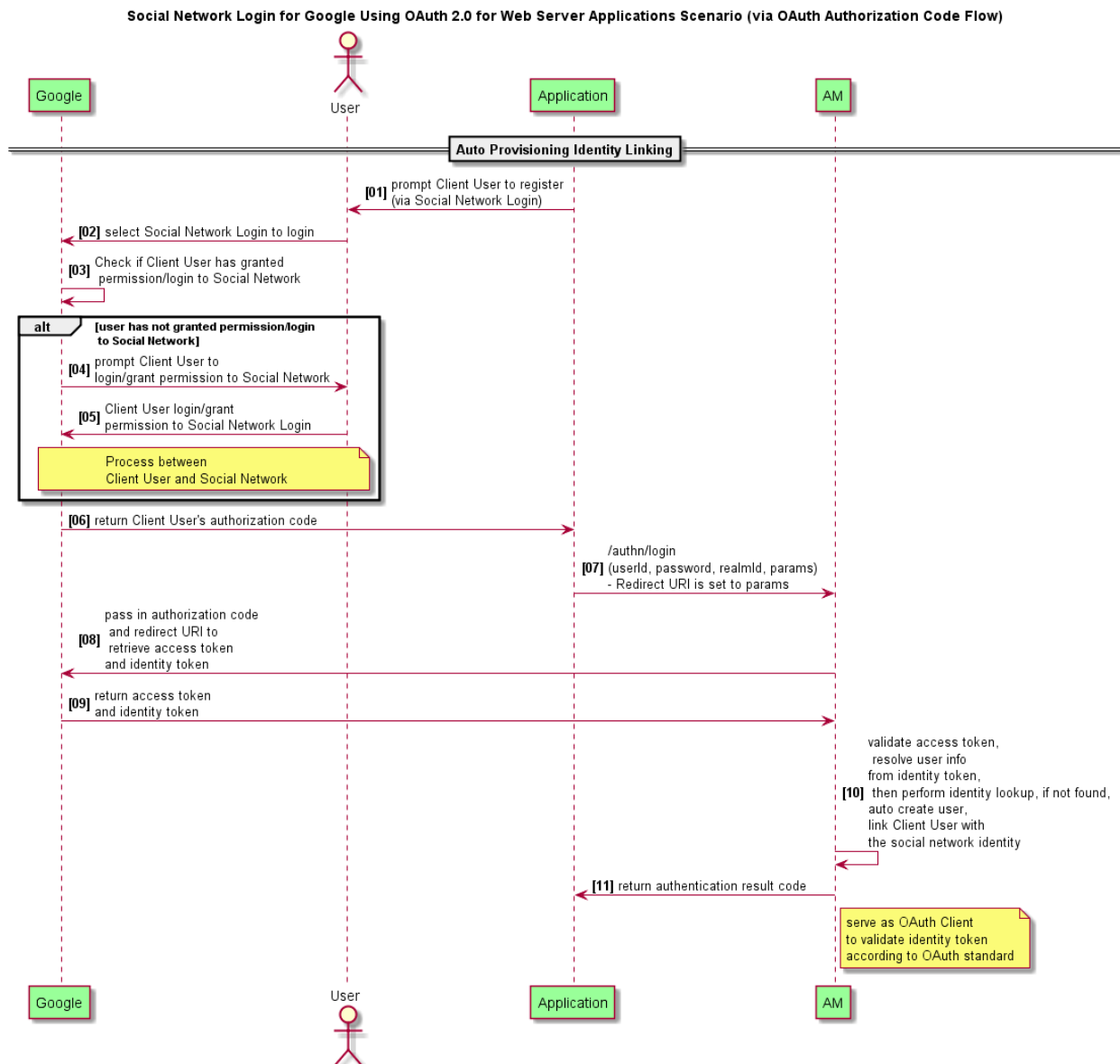
REST Sample Code

This is a sample REST API call for Google Sign-In using Google Sign-In SDK (via OAuth Implicit Flow).

Login via Google Sign-In Using Google Sign-In SDK (via OAuth Implicit Flow)	
URL: /authn/login	
Request	
JSON	<pre>{ "userId": "socialGoogle", "password": "eyJhbGciOiJSUzI1NiIsImtp....<truncated in sample>.....9bRH9vE5- jVWHFoBGS163VXfgzxU0U_Zm9g12fk__FSzBHxX3_jRSi5f0NXJw", "realmId": "google" }</pre>
Response	
JSON	<pre>{ "result": { "authenticated": true, "sessionToken": "10001_ELns3HSnKM3Kju9RuGk8pqp....<truncated in sample>.....a- 7HCiSnx0xhlGuIjd8YO8bg3HLWHp6yxvDLH6L0OKXKkDsqtg", ... } }</pre>

OAuth 2.0 for Web Server Applications Scenario (via OAuth Authorization Code Flow)

Sequence Diagram



REST Sample Code

This is a sample REST API call for Google Sign-In using OAuth 2.0 for Web Server Applications Scenario (via OAuth Authorization Code Flow).

Login via Google Sign-In Using OAuth 2.0 for Web Server Applications Scenario (via OAuth Authorization Code Flow)

URL: /authn/login

Request

JSON

```
{
  "userId": "socialGoogle",
  "password": "4/ud4OC5fRI5cmWvkej.....<truncated in sample>.....igI9kvISvTylSA",
  "realmId": "google",
  "params": {
    "redirect_uri": "http://dev.i-sprint.demo.com:8081/"
  }
}
```

Response

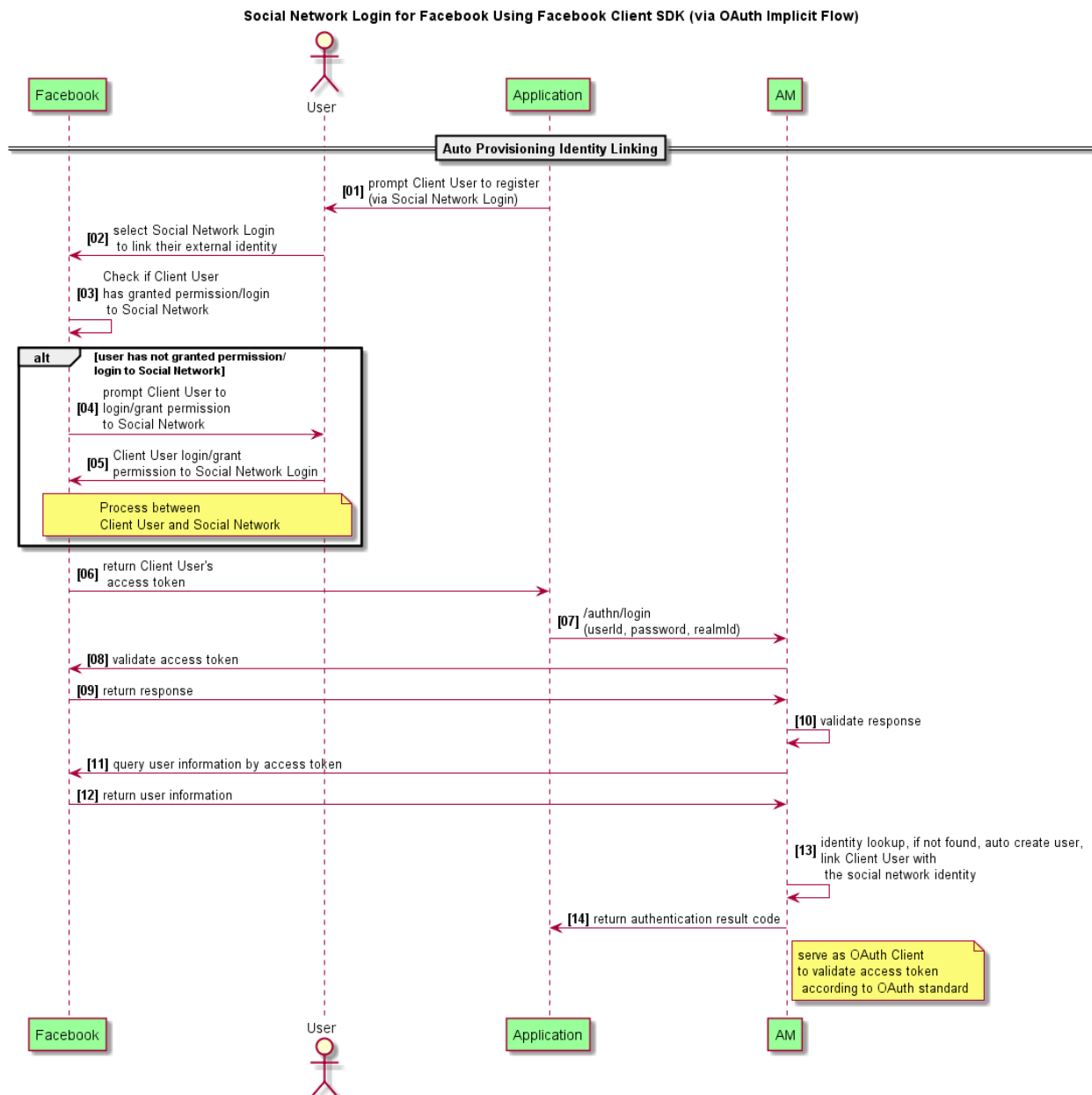
JSON

```
{
  "result": {
    "authenticated": true,
    "sessionToken":
"10001_ELns3HSnKM3Kju9RuGk8pqp.....<truncated in sample>.....a-7HCiSnx0xhlGuIjd8YO8bg3HLWHp6yxvDLH6L0OKXKkDsqtg",
    ...
  }
}
```

Facebook

Using Facebook Client SDK (via OAuth Implicit Flow)

Sequence Diagram



REST Sample Code

This is a sample REST API call for Facebook Login using Facebook Client SDK (via OAuth Implicit Flow).

Login via Facebook Login Using Facebook Client SDK (via OAuth Implicit Flow)

URL: /authn/login

Request

JSON

```
{
  "userId": "socialFacebook",
  "password": "EAAFBOVqRiCABALik0i.....<truncated in
sample>.....kjBZCeFp6",
  "realmId": "facebook"
}
```

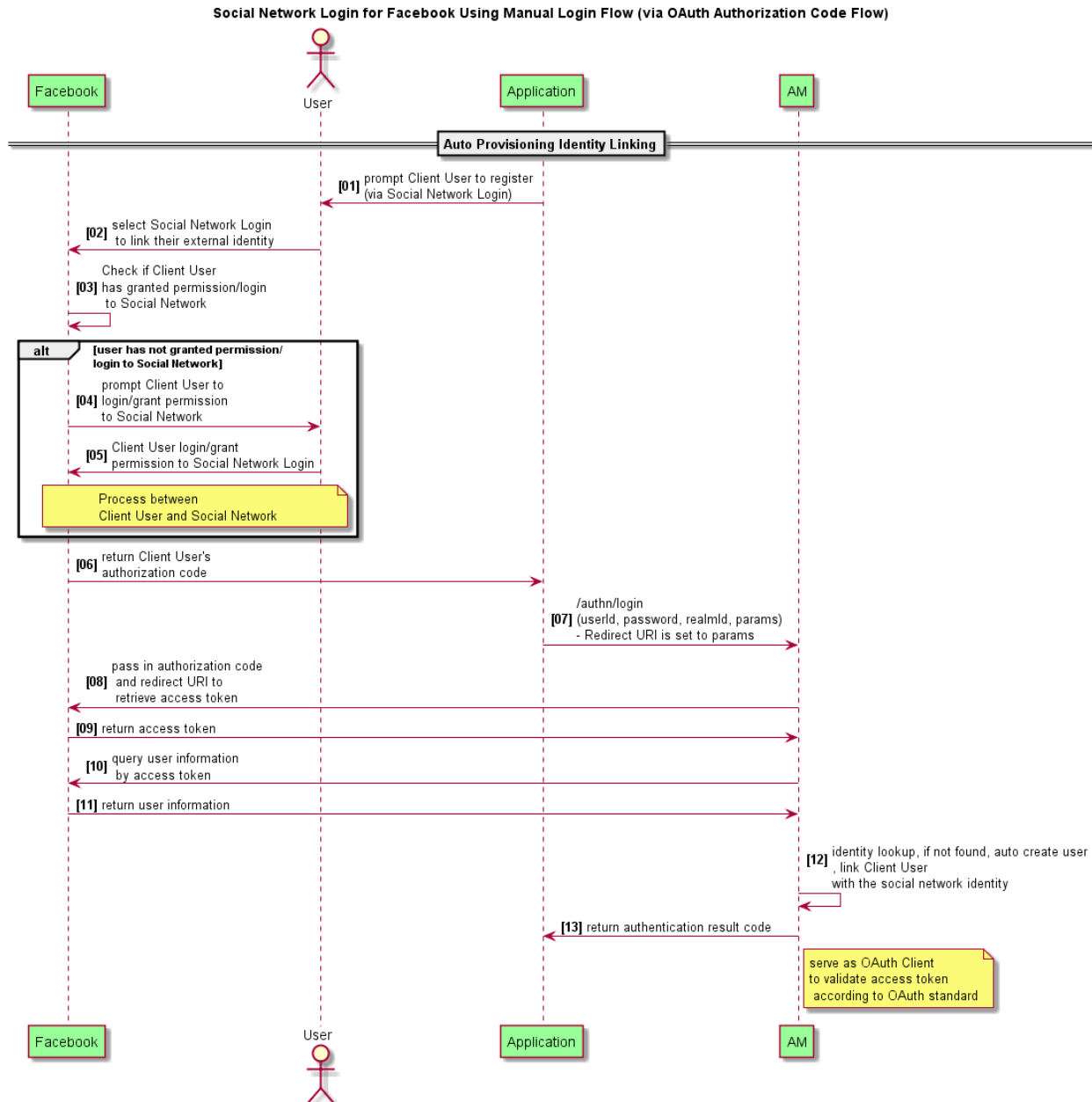
Response

JSON

```
{
  "result": {
    "authenticated": true,
    "sessionToken":
"10001_Elns3HSnKM3Kju9RuGk8pqp.....<truncated in
sample>.....a-
7HCiSnx0xhlGuIjd8YO8bg3HLWHP6yxvDLH6L0OKXKkDsqtg",
    ...
  }
}
```

Manual Login Flow (via OAuth Authorization Code Flow)

Sequence Diagram



REST Sample Code

This is a sample REST API call for Facebook Login via Manual login flow (using OAuth Authorization Code Flow).

Login via Facebook Login via Manual login flow (using OAuth Authorization Code Flow)

URL: /authn/login

Request

JSON

```
{
  "userId": "socialFacebook",
  "password": "AQA4Tamq_WfGyaf-3li8nRZxyd.....<truncated in sample>.....0eR3sGE6bbk6GvWnpcdKCECf#_=_",
  "realmId": "facebook",
  "params": {
    "redirect_uri": "http://fb.i-sprint.demo:8081/fbdemo/"
  }
}
```

Response

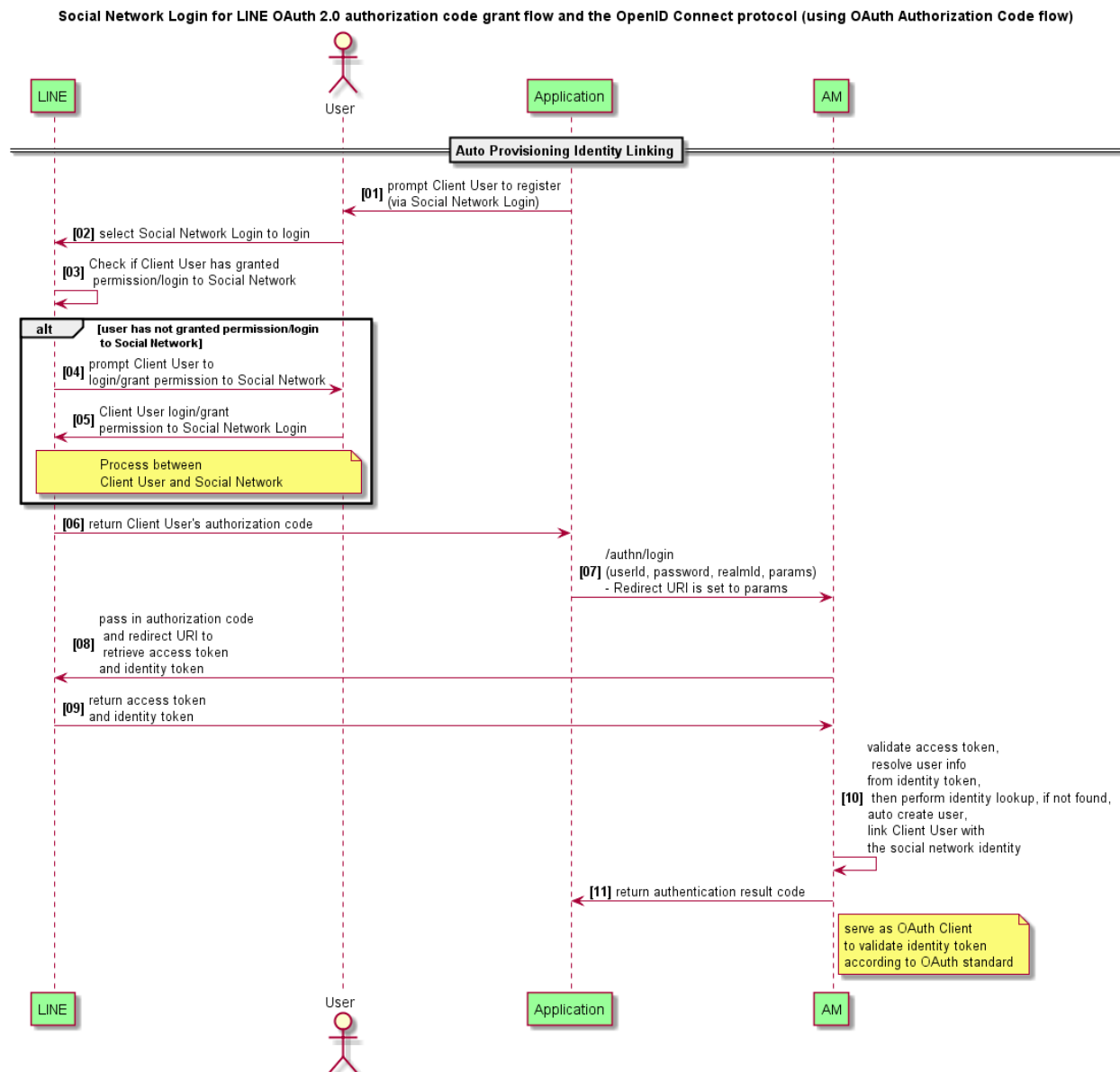
JSON

```
{
  "result": {
    "authenticated": true,
    "sessionToken":
"10001_ELns3HSnKM3Kju9RuGk8pqp.....<truncated in sample>.....a-7HCiSnx0xhlGuIjd8YO8bg3HLWHP6yxvDLH6L0OKXKkDsqtg",
    ...
  }
}
```


LINE

OAuth 2.0 Authorization Code Grant Flow and the OpenID Connect Protocol (via OAuth Authorization Code Flow)

Sequence Diagram



REST Sample Code

This is a sample REST API call for LINE OAuth 2.0 authorization code grant flow and the OpenID Connect protocol (using OAuth Authorization Code flow).

Login via LINE OAuth 2.0 authorization code grant flow and the OpenID Connect protocol (using OAuth Authorization Code flow)

URL: /authn/login

Request

JSON

```
{
  "userId": "socialLINE",
  "password": "4/ud4OC5fRI5cmWvkej.....<truncated in
sample>.....igI9kvISvTylSA",
  "realmId": "LINE",
  "params": {
    "redirect_uri": "http://dev.i-
sprint.demo.com:8081/linedemo"
  }
}
```

Response

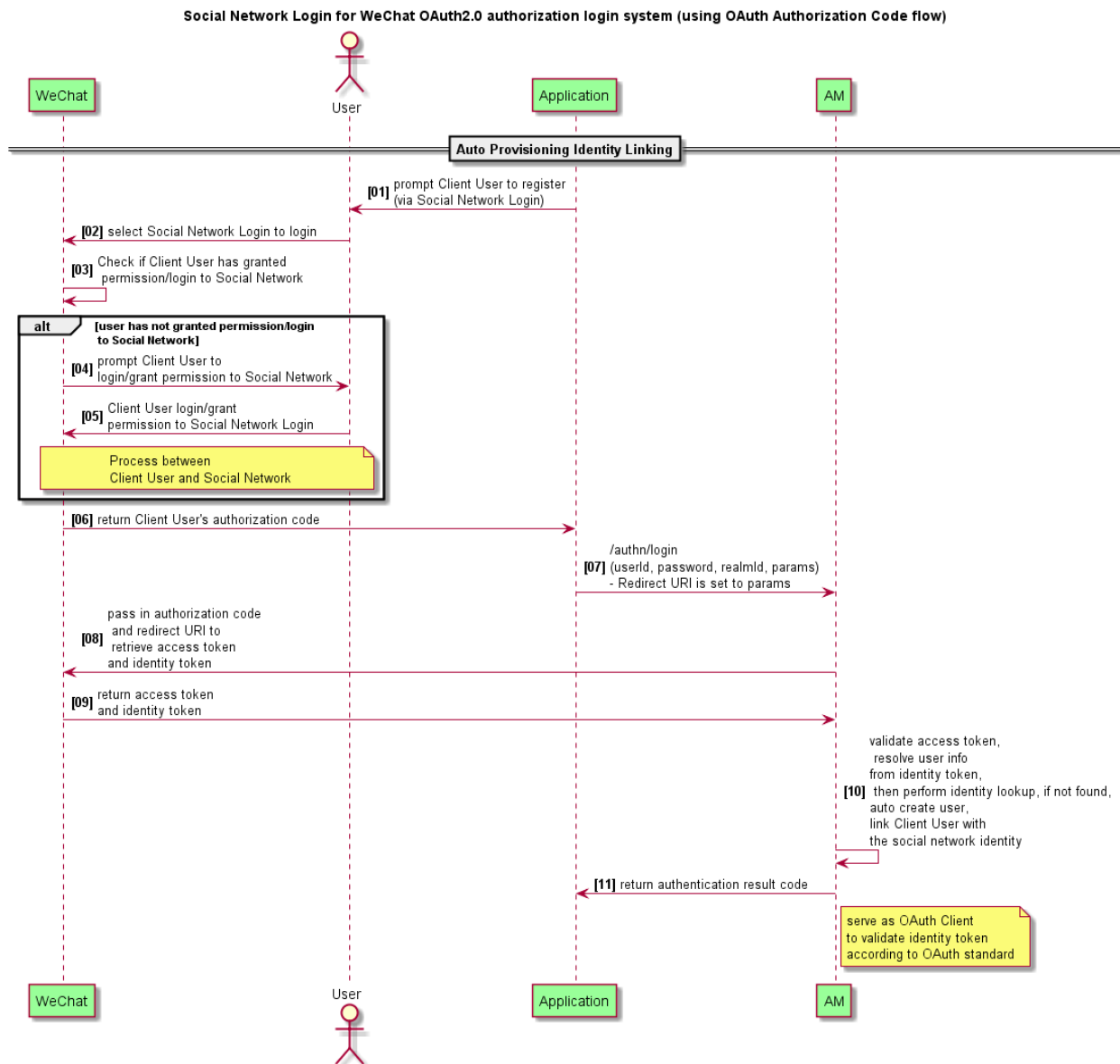
JSON

```
{
  "result": {
    "authenticated": true,
    "sessionToken":
"10001_ELns3HSnKM3Kju9RuGk8pqp.....<truncated in
sample>.....a-
7HCiSnx0xhlGuIjd8YO8bg3HLWHp6yxvDLH6L0OKXKkDsqtg",
    ...
  }
}
```

WeChat

OAuth 2.0 Authorization Login System (Using OAuth Authorization Code Flow)

Sequence Diagram



REST Sample Code

This is a sample REST API call for WeChat OAuth 2.0 authorization login system (using OAuth Authorization Code flow).

Login via WeChat OAuth 2.0 authorization login system (using OAuth Authorization Code flow)	
URL: /authn/login	
Request	
JSON	
<pre>{ "userId": "socialWeChat", "password": "4/ud4OC5fRI5cmWvkej.....<truncated in sample>.....igI9kvISvTylSA", "realmId": "WECHAT", "params": { "redirect_uri": "http://dev.i-sprint.demo.com:8081/" } }</pre>	
Response	
JSON	
<pre>{ "result": { "authenticated": true, "sessionToken": "10001_ELns3HSnKM3Kju9RuGk8pqp.....<truncated in sample>.....a-7HCiSnx0xhlGuIjd8YO8bg3HLWHP6yxvDLH6L0OKXKkDsqtg", ... } }</pre>	

Parameters & Exceptions

This method is used to log in with Social Network user identity:

- password is a token given by Social Network upon successful login, e.g. access_token, authorization code.
- userId is not in use; however, it cannot be an empty or null value.
- params is the map for extra fields to be passed in, e.g. redirect_uri, state.
- realmId is the AuthenticationRealm Id which is comprised of Social Network Lookup module (e.g. Facebook Lookup Module, Google Lookup Module, LINE Lookup Module & WeChat Lookup Module) and Social Network Login Module.

For exception and other details of the API method, refer to the [AccessMatrix REST API Specification](#).

[Click here](#) to view a summary of fields specifically required in different implementations of Facebook, Google, LINE and WeChat.

Responses

AuthnResultTO is the return value of this API where its responses consist of attributes pertaining to the user info and the corresponding token(s) passed by the caller.

Below are the attributes/fields of the corresponding social network applications and auto-create user response:

- **Google:**
 - token - access_token, id_token,
 - attributes - email, sub, name, hd, picture, locale
- **Facebook:**
 - token - access_token
 - attributes - email, id, name
- **LINE:**
 - token - access_token, id_token
 - attributes - iss, sub, aud, exp, iat, nonce, name, picture, email
- **WeChat:**
 - token - access_token
 - attributes - openid, nickname, sex, province, city, country, headimgurl, privilege, unionid

Refer to the [AccessMatrix REST API Specification](#) for details.

4.4. Auto-Provisioning

The auto-Provisioning feature allows social network users to be created in AccessMatrix server automatically if the users do not exist in AM5 server yet. Subsequently, the users are also linked with the corresponding social network identity.

To allow this feature, the corresponding social network lookup module needs to be configured to enable the **Auto Create New User** attribute. The new user will be created in the Default Store of AccessMatrix.

Below is the mapping of the social network user profile attributes and AM5 user attributes:

Facebook:

- User ID mapping can be configured either to use Facebook ID or Facebook email on the Admin Console. Facebook email is the default ID to be mapped to the AM5 user Id.
(When "Facebook Email" is specified as **User Id Mapping**, user email must be registered on the Facebook user profile, otherwise, AM5 is going to trigger error response for missing email. When "Facebook ID" is specified as **User Id Mapping**, AM5 is still going to map the Facebook email to AM5 User ID if it is available, and Facebook ID will be used to map to the AM5 user Id only when the user email is not registered. Meanwhile, the requester APP can override **User Id Mapping** by providing "userId" as an additional param in the API call).
- Facebook user name is mapped to an AM5 user name.

Google :

- Google user email is mapped to AM5 user Id and user email.
- Google user name is mapped to an AM5 user name.

LINE:

- User ID Mapping can be configured either to use LINE ID or LINE email on Admin Console. LINE email is the default ID to be mapped to AM5 user Id.
(When "LINE Email" is specified as **User Id Mapping**, user email must be registered on a LINE user profile, otherwise, AM5 is going to trigger an error response for missing email. When "LINE ID is specified" as **User Id Mapping**, AM5 is still going to map the LINE email to the AM5 user Id if it is available, LINE ID will be used to map to the AM5 user Id only when the user email is not

registered. Meanwhile, the requester APP can override User ID Mapping by providing "userId" as an additional param.)

- LINE user name is mapped to an AM5 user name.

WeChat:

- WeChat user id is mapped to AM5 user Id.
- WeChat user nickname is mapped to AM5 username.



Note:

Facebook allows:

- user's email to be used for registration, even though user email can be modified
- user's mobile to be used for registration and login

Alternatively during user creation, user Id can be:

- defined and passed in via login API (using "userId" as the key for params)
- or use the Facebook Id if the email is not found after querying the user profile from Facebook

REST Sample Code

This is a sample REST API call of Auto Provisioning with identity linking for Google using Google Sign-In SDK (via OAuth Implicit Flow).

Login via Google after enable "Auto Create New User" attribute

URL: /authn/login

Request

JSON

```
{
  "userId": "dummyId",
  "password": "eyJhbGciOiJSUzI1NiIsImtp....<truncated in
sample>.....9bRH9vE5-
jVWHFoBGS163VXfgzxU0U_Zm9g12fk__FSzBHxX3_jRSi5f0NXJw",
  "realmId": "google"
  "params": {
    "userId": "Jude"
  }
}
```

Login via Google after enable "Auto Create New User" attribute

Response

JSON

```
{
  "result": {
    "authenticated": true,
    "sessionToken":
    "10001_ELns3HSnKM3Kju9RuGk8pqp.....<truncated in
    sample>.....a-
    7HCiSnx0xhlGuIjd8YO8bg3HLWHP6yxvDLH6L0OKXKkDsqtg",
    "userId": "Jude"
    ...
  }
}
```

Parameters & Exceptions

This method is used to auto-create AM5 user and links the social network identity to it.

- `userId` - this field does not refer to any user and it is not used, any value can be set to it.
- `password` - the token given by Social Network upon successful login (mandatory field).
- `profileName` - used to identify the identity mapping of user and external Social Network ID. It is not configurable for the auto-provisioning use case. The default value will be used.
- `params` - the map for extra fields to be passed in.
 - `realmId` is the Authentication Realm ID of Social Network which includes Social Network Lookup module, e.g. Facebook Login Lookup Module, Google Sign-In Lookup Module, and Social Network Login module. (mandatory field)
 - Optionally, user Id of AM5 user to be auto-created can be passed in using `params`, with `"userId"` as the param's key.

For exception and other details of the API method, refer to the [AccessMatrix REST API Specification](#).

[Click here](#) to view a summary of fields specifically required in different implementations of Facebook and Google.

Responses

Returns a JSON object from this API. Its response field may consist of :

- `externalId` - the identifier of Social Network login user. This is used by the AccessMatrix server for identity linking.

-
- Below are the attributes/fields of the corresponding social network applications and auto-create user response:
 - **Google:**
 - token - access_token, id_token,
 - attributes - email, sub, name, hd, picture, locale
 - **Facebook:**
 - token - access_token
 - attributes - email, id, name
 - **LINE:**
 - token - access_token, id_token
 - attributes - email, id, name
 - **WeChat:**
 - token - access_token
 - attributes - openid, nickname, sex, province, city, country, headimgurl, privilege, unionid
 - **Indicator of whether user is auto-created (if applicable)**
 - "user_auto_created" is 'true'
-

4.5. Display Listing of User's Linked Social Network(s)

REST Sample Code

This is a sample REST API call to display listing of user's linked Social Network(s).

Get Social Network Mapped ID Client details	
URL: /authz/social/mappedIdByExtAppId/get	
Request	
JSON	<pre>{ "extAppId": "google", "userId": "karen.tan@yahoo.com", "sessionToken": "10001_ELns3HSnKM3Kju9RuGk8pqp.....<truncated in sample>.....a- 7HCiSnx0xhlGuIjd8YO8bg3HLWHP6yxvDLH6L0OKXKkDsqtg" }</pre>
Response	
JSON	<pre>{ "result": { "profileName": "My Gmail Profile", "extAppId": "google", "extAppUUID": "GOOGLE", "userUUID": "TestJdbc:Ngz6laEEE_", "extId": "mratkyaw@gmail.com", "extIdType": "Email" } }</pre>

Parameters & Exceptions

This method is used to list the linked Social Network of a user based on AM5 user identity:

- `userId` is an AM5 user identity.
- `extAppId` is the ID of Social Network Login Lookup module, e.g. Facebook Lookup Module, Google Lookup Module, LINE Lookup Module and WeChat Lookup Module (mandatory field).

For this method, the `userId` field is mandatory.

For exception and other details of the API method, refer to the *UAM Externalized Authorization & SAML API Reference*.

Response

Returns a JSON object from this API. It consists of `extAppId` (lookup module Id), `extAppUUID` (lookup module UUID), `extId` (Social Network identity used for identity mapping in AM5), `profile`, `extIdType` (Type of Social Network Identity, e.g. FacebookId for Facebook Login, Email for Google Sign-In, LINEId for LINE Login and WeChatId for WeChat Login), and user UUID.

5. Appendices

This guide contains the following appendices:

- [A. Allowing Email as User Id in AccessMatrix](#)
- [B. Self-Service Linking Sample Program](#)
- [C. Summary of Fields Required in Social Network Login](#)

5.1. A. Allowing Email as User Id in AccessMatrix

For auto-provisioning, email may be used as the user Id. Thus, AccessMatrix **Global Setting** needs to be configured to allow characters used in email addresses.

1. Navigate to **Administration Dashboard > Configuration > Global Setting > Primary Key Management** tab.
2. Click **Edit** on the **View Global Setting** page.
3. Modify the **User/Group Id Charset Restriction attribute** to "[a-zA-Z0-9@.\- _]+".
4. Modify the **Max User/Group Id Length** to "40" characters, or any other required value.

5.2. B. Self-Service Linking Sample Program

This section illustrates the use cases, setup and deployment guide of Social Network Login sample programs: **AccessMatrix-version-socialnetworklogin-implicit.war** and **AccessMatrix-version-socialnetworklogin-authcode.war**.



Notes:

- In AccessMatrix version 5.5.2, **AccessMatrix-version-socialnetworklogin-implicit.war** is named as **AccessMatrix-version-socialnetworklogin.war**.
- The **AccessMatrix-version-socialnetworklogin-authcode.war** WAR file is only available from AccessMatrix version 5.6.0 onwards.

B1. Purpose

The Social Network Login sample programs are designed to demonstrate OAuth implicit flow and OAuth authorization code flow of:

- **Facebook Login** using:
 - Facebook Client SDK (OAuth Implicit flow)
 - Manual Login Flow (OAuth Authorization Code Flow)
- **Google Sign-In** using:
 - Google Sign-In SDK (OAuth Implicit flow)
 - OAuth 2.0 for Web Server Applications scenario (OAuth Authorization Code flow)
- **LINE Login** using:
 - OAuth 2.0 authorization code grant flow and the OpenID Connect protocol (OAuth Authorization Code flow)
- **WeChat Login** using:
 - OAuth2.0 authorization login system (OAuth Authorization Code flow)

In the sample program, Social Network Login is introduced as the alternative authentication method for existing AccessMatrix users who have been using an existing One-Factor Authentication (1FA) with AccessMatrix UAS. To activate Social Network Login as the login alternative, end users need to be authenticated first using 1FA followed by triggering the Social Network Login. This will allow the external identities to be linked up with the AM5 user identities. AM5 user is only allowed to perform external identity linking once, otherwise, an error will be thrown.

B2. Use Cases

Below are the use cases and the corresponding data flow diagrams for Facebook Login and Google Sign-In:

OAuth Implicit Flow

WAR file name: **socialnetworklogin-implicit.war**

- **Google**

To find out the details of use cases of Google Sign-In, refer to:

- [Self Service Identity Linking](#) for mapping of Google Identity with AM5 user.
- [Google Sign-In \(using Google Sign-In SDK\)](#) for implementation of Google Sign-In.

- **Facebook**

To find out the details of more use cases of Facebook Login, refer to :

- [Self Service Identity Linking](#) for mapping of Facebook Identity with AM5 user.
- [Facebook Login \(using Facebook Client SDK \)](#) for implementation of Facebook Login.

OAuth Authorization Code Flow

WAR file name: **socialnetworklogin-authcode.war**

- **Google**

To find out the details of use cases of Google Sign-In, refer to:

- [Self Service Identity Linking](#) for mapping of Google Identity with AM5 user.
- [Google Sign-In \(using OAuth 2.0 for Web Server Applications scenario\)](#) for implementation of Google Sign-In.

- **Facebook**

To find out the details of more use cases of Facebook Login, refer to :

- [Self Service Identity Linking](#) for mapping of Facebook Identity with AM5 user.
- [Facebook Login \(using Manual Login Flow\)](#) for implementation of Facebook Login.

- **LINE**

To find out the details of more use cases of LINE Login, refer to :

- [Self Service Identity Linking](#) for mapping of LINE Identity with AM5 user.
 - [LINE Login \(using OAuth 2.0 authorization code grant flow and the OpenID Connect protocol\)](#) for implementation of LINE Login.
-

- **WeChat**

To find out the details of more use cases of WeChat Login, refer to :

- [Self Service Identity Linking](#) for mapping of WeChat Identity with AM5 user.
- [WeChat Login \(Using OAuth2.0 authorization login system\)](#) for implementation of WeChat Login.

B3. Setup

- **AccessMatrix Integration With Social Network Application**

Refer to [Social Network Login Configuration](#) to find out the setup and configurations details required for the integration of the respective social network application and AccessMatrix UAM.

- **Configuration and Setup Guide of Social Network JavaScript Client Application**

Refer to the **readme.txt** of **socialnetworklogin-implicit.war** or **socialnetworklogin-authcode.war** for the configurations and deployment guide of the sample program.

B4. Reference Guides

- **Google Sign-In Integration**

The JavaScript sample code of Google Sign-In is available in the sample program. For more details, refer to the reference guide in [Google Developer documentation](#).

- **Facebook Login Integration**

The JavaScript sample code of Facebook Login is available in the sample program. For more details, refer to the reference guide in [Facebook Developer documentation](#).

- **LINE Login Integration**

The JavaScript sample code of LINE Login is available in the sample program. For more details, refer to the reference guide in [LINE Developer documentation](#).

- **WeChat Login Integration**

The JavaScript sample code of WeChat Login is available in the sample program. For more details, refer to the reference guide in [WeChat Developer documentation](#).

5.3. C. Summary of Fields Required in Social Network Login

Below is a summary of fields specifically required in different Social Network Login Implementations (for Login and Social Identity Linking APIs):

Fields	Facebook		Google		LINE	WeChat
	Facebook Client SDK (using OAuth Implicit flow)	Manual login flow (using OAuth Authorization Code flow)	Using Google Sign-In SDK (using OAuth Implicit flow)	OAuth 2.0 for Web Server Applications scenario (using OAuth Authorization Code flow)	OAuth 2.0 authorization code grant flow and the OpenID Connect protocol (using OAuth Authorization Code flow)	OAuth2.0 authorization on login system (using OAuth Authorization Code flow)
tokenToBeResolved	access token	authorization code	identity token	authorization code	authorization code	authorization code
accessToken	No	No	Yes	No	No	No
redirectURI	-	Yes	-	Yes	Yes	Yes



Notes:

- tokenToBeResolved - The name of the token given by Social Network upon successful login, where it is used to resolve user info at the social network application. (mandatory field).
- accessToken - Set "access_token" as the key with its value to params. It is used by Google to validate the integrity of the identity token. This field is optional as it will grant access to the user account which allows calling Google APIs on behalf of the user. So far, AccessMatrix server does not make use of this token for such purpose.
- redirectURI - Set "redirect_uri" as the key with its value to params. It is used by Social Network App to verify the validity of the client app.
